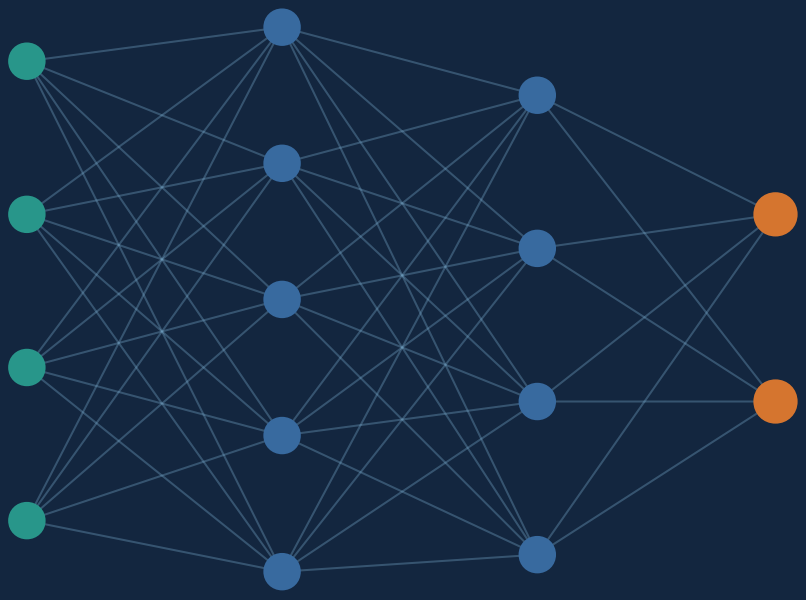


BOOK CHAPTER · 2026

Practical Applications of Machine Learning and Deep Learning in Healthcare and Financial Data Analytics

Five implementation case studies: PCOS analytics, fungal microscopy, skin disease detection, colon and lung cancer histopathology, and Google stock forecasting

Luxshi Karunakaran

Department of Social Statistics, University of Sri Jayewardenepura, Sri Lanka
B.Sc. (Hons) in Computer Science and Technology · M.Sc. in Data Science and Artificial Intelligence (reading)

github.com/luxshikarunakaran

October 2026

Abstract

This chapter presents five implementations of machine learning (ML) and deep learning (DL) across healthcare and finance: polycystic ovary syndrome (PCOS) analysis from clinical and survey records, fungal microscopy classification, mobile skin disease detection, colon and lung cancer histopathology classification, and Google stock price forecasting. After reviewing the theory each project relies on, it describes a shared, leakage-aware methodology and reports each project as a structured case study.

On PCOS data, eight synthetic-data generators gave no significant gain over an unaugmented baseline (clinical ROC-AUC 0.946), while unsupervised K-Means recovered clinical PCOS with 0.810 balanced accuracy. On DeFungi, a fine-tuned ConvNeXtV2 reached 85.1% accuracy and macro-F1 0.883, and a leakage audit exposed a 12-point optimism gap in random splits. On LC25000, Xception reached 100% test accuracy and ResNet50 99.4%. GRU–XGBoost hybrids reached test R^2 0.987 on GOOGL prices but MAPE above 40%, traced to unadjusted stock splits and indicator leakage.

A comparative analysis shows that evaluation design — grouped and chronological splits, variance reporting, calibration and naive baselines — changed conclusions more than model choice. The chapter closes with ethical considerations and directions for externally validated, explainable and deployable AI.

machine learning · deep learning · transfer learning · synthetic data · data leakage · explainable AI · medical image classification · time-series forecasting

AUTHOR

Luxshi Karunakaran
Department of Social Statistics
University of Sri Jayewardenepura, Nugegoda, Sri Lanka
B.Sc. (Hons) Computer Science and Technology
M.Sc. Data Science and Artificial Intelligence (reading)

HOW TO CITE

Karunakaran, L. (2026). *Practical Applications of Machine Learning and Deep Learning in Healthcare and Financial Data Analytics* [Book chapter]. Department of Social Statistics, University of Sri Jayewardenepura.

CODE AND DATA

github.com/luxshikarunakaran
Implementation-in-PCOS
Fung-implementation
skin-disease-detection-mobile-app
Colon-lung-cancer-image-data
Google-stock-analysis

© 2026 Luxshi Karunakaran. Figures credited to a repository or to the SICET 2025 paper are reproduced from the author's own work; all other diagrams and charts were prepared for this chapter from the reported results.

Contents

01	Introduction	4
02	Theoretical Foundations of Machine Learning	6
03	Theoretical Foundations of Deep Learning	8
04	Data Preparation and Experimental Methodology	10
05	Case Study I: Machine Learning for PCOS Data Analysis	14
06	Case Study II: Deep Learning for Fungal Image Classification	21
07	Case Study III: Deep Learning for Skin Disease Detection	26
08	Case Study IV: Deep Learning for Colon and Lung Cancer Image Classification	29
09	Case Study V: Machine Learning for Google Stock Analysis	37
10	Comparative Analysis Across the Five Projects	43
11	Ethical, Privacy and Responsible AI Considerations	45
12	Conclusion and Future Directions	46
13	References and Supplementary Materials	48

How to read this chapter. Sections 1–4 give the theory and a shared methodology. Sections 5–9 are self-contained case studies; each opens with a **Key result** box and follows the same structure — problem, data, method, results, limitations. Section 10 compares the five projects, and Sections 11–12 cover responsible AI and future directions.

01 Introduction

This chapter shows how one practitioner applied machine learning (ML) and deep learning (DL) to five real problems: three in medical imaging, one in clinical tabular data, and one in financial time series. Each project is documented in a GitHub repository and written up here as a reproducible case study.

1.1 Background of Artificial Intelligence, Machine Learning and Deep Learning

Artificial intelligence (AI) is the broad field of building systems that perform tasks normally requiring human judgement. Machine learning is the subset of AI in which a system learns a mapping from data rather than following hand-written rules. Deep learning is the subset of ML that learns layered representations with multi-layer neural networks.

Classical ML works well on structured, tabular data where features are already meaningful (age, BMI, hormone levels, closing price). Deep learning dominates on unstructured data such as images, audio and text, because convolutional and attention-based networks learn the features themselves.

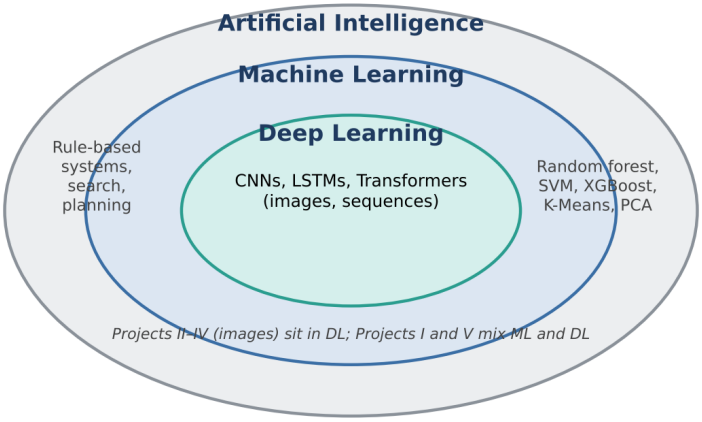


Figure 1. Relationship between artificial intelligence, machine learning and deep learning.

1.2 Importance of data-driven prediction and classification

Prediction estimates a future or unknown numeric value; classification assigns an input to one of a fixed set of categories. Both turn historical records into decisions that can be repeated at scale, audited and improved.

In healthcare, a classifier can triage images or flag at-risk patients so that specialists spend time where it matters. In finance, a forecaster gives a quantitative baseline against which human judgement can be compared.

1.3 Applications in healthcare and financial analytics

Healthcare applications covered in this chapter are: syndrome detection from clinical and survey records (PCOS), fungal species identification from microscope images, skin disease recognition on a mobile phone, and cancer detection from histopathology slides. The financial application is analysis and forecasting of Google (Alphabet, GOOGL) share prices.

The two domains differ in one important way. Medical labels are largely stationary (a malignant tissue pattern does not change next year), while stock prices are non-stationary and influenced by events the data cannot see.

1.4 Chapter objectives and scope

The chapter has four objectives:

1. Summarise the ML and DL theory needed to understand the five implementations.
2. Describe a common, leakage-aware data and experiment methodology.
3. Present each project as a structured case study: problem, data, method, implementation, results, limitations.
4. Compare the projects and draw lessons on model choice, evaluation and responsible AI.

The scope is applied: theory is included only where it explains a design decision in the code. Reported numbers come from the project repositories and the associated papers.

1.5 Overview of the five implementation projects

Table 1. Overview of the five implementation projects

#	Project	Domain	Data type	Main approach	Repository
I	PCOS data analysis	Healthcare	Tabular (clinical + survey)	Supervised and unsupervised ML, PCA, synthetic data	Implementation-in-PCOS
II	Fungal image classification	Healthcare	Microscopy images	CNN / transformer features, transfer learning, fusion	Fung-implementation
III	Skin disease detection app	Healthcare	Clinical photographs	CNN transfer learning deployed on mobile	skin-disease-detection-mobile-app
IV	Colon and lung cancer	Healthcare	Histopathology images	Pretrained CNNs, fine-tuning	Colon-lung-cancer-image-data
V	Google stock analysis	Finance	Daily time series	Regression and forecasting with hyperparameter tuning	Google-stock-analysis

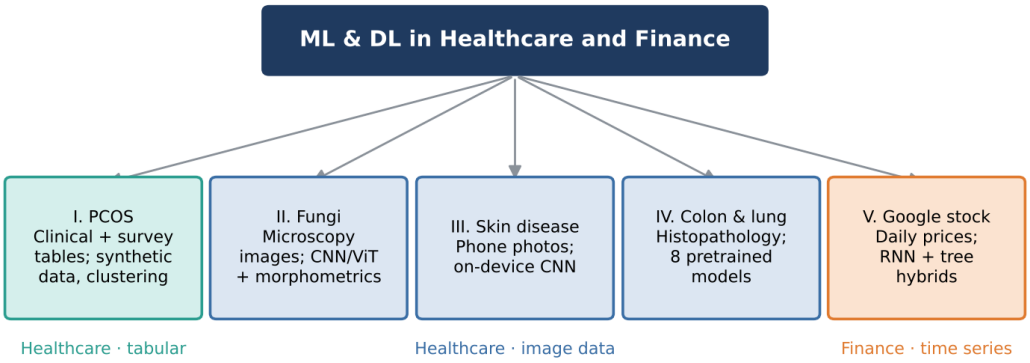


Figure 2. The five implementation projects covered in this chapter, grouped by domain and data type.

02 Theoretical Foundations of Machine Learning

Machine learning fits a function $f(x) \approx y$ from examples, chosen so that it generalises to inputs it has not seen. Every project in this chapter is an instance of that idea; they differ in what x is (a table row, an image, a price history) and what y is (a class, a cluster, a number).

2.1 Definition and principles of Machine Learning

Mitchell's classic definition: a program learns from experience E with respect to task T and performance measure P if its performance on T, measured by P, improves with E. In practice, E is a dataset, T is a prediction task and P is a metric such as accuracy or RMSE.

Three principles govern every model. Generalisation matters more than training fit. Simpler models are preferred when performance is equal (Occam's razor). And the evaluation data must stay independent of every decision made during training.

2.2 Types of Machine Learning

Table 2. Types of Machine Learning

Type	Labels available	Goal	Used in this chapter
Supervised	Yes	Predict y from x	All five projects
Unsupervised	No	Find structure in x	PCOS clustering and PCA
Semi-supervised	Few	Use unlabelled data to help a small labelled set	Discussed as future work
Self-supervised	Generated from x	Learn representations without manual labels	Pretrained backbones (indirectly)
Reinforcement	Reward signal	Learn a policy by interaction	Not used

2.3 Supervised Learning

Given labelled pairs (x_i, y_i), a supervised learner minimises a loss $L(f(x_i), y_i)$ over the training set. Common tabular learners used in the projects include logistic regression, k-nearest neighbours, support vector machines, decision trees, random forests and gradient boosting (XGBoost, LightGBM).

Ensembles such as random forests reduce variance by averaging many decorrelated trees. Gradient boosting reduces bias by fitting each new tree to the residual errors of the previous ones.

2.4 Unsupervised Learning

Unsupervised learning works on x alone. Its outputs are groupings (clustering), lower-dimensional coordinates (dimensionality reduction), or density estimates. In clinical data, it is used to discover phenotypes that the diagnostic label does not capture.

2.5 Classification and Regression

Classification predicts a discrete label: PCOS vs no PCOS, five tissue classes in LC25000, fungal species. Regression predicts a continuous value: the next day's closing price of GOOGL. The loss and the metric follow from this choice — cross-entropy and F1 for classification, squared error and RMSE for regression.

2.6 Clustering and Dimensionality Reduction

K-means partitions data into k groups by minimising within-cluster squared distance. Gaussian mixture models allow soft, elliptical clusters. Agglomerative clustering builds a hierarchy by merging the closest groups. DBSCAN finds dense regions and labels sparse points as noise.

Principal component analysis (PCA) projects data onto orthogonal directions of maximum variance:

$$Z = XW_k, \quad W_k = \text{top } k \text{ eigenvectors of } \tfrac{1}{n}X^\top X$$

PCA is used both to visualise high-dimensional data in two dimensions and to remove redundant, correlated features before modelling. t-SNE and UMAP are non-linear alternatives that preserve local neighbourhoods better but distort global distances.

2.7 Feature Engineering and Feature Selection

Feature engineering creates informative inputs from raw data: ratios such as LH/FSH in PCOS, lagged prices and moving averages in stock data. Feature selection removes inputs that add noise or leak the answer.

Selection methods fall into three families. Filter methods rank features by a statistic (chi-square, mutual information, correlation). Wrapper methods search subsets using model performance (recursive feature elimination). Embedded methods select during training (L1 regularisation, tree importance).

2.8 Training, Validation and Testing

Data are split into three roles. The training set fits parameters. The validation set chooses hyperparameters and decides when to stop. The test set is touched once, at the end, to estimate real-world performance.

K-fold cross-validation rotates the validation role across k folds and averages the score, which gives a more stable estimate on small datasets such as the 541-patient PCOS clinical set. Stratified folds keep class proportions equal in every fold. For time series, folds must respect time order (Section 4.7).

2.9 Overfitting, Underfitting and Regularization

An overfitted model memorises training noise: training error is low but validation error is high. An underfitted model is too simple to capture the pattern: both errors are high. The bias–variance trade-off describes the balance.

Regularisation limits model complexity. L2 (ridge) shrinks weights towards zero; L1 (lasso) drives some weights exactly to zero. Trees are regularised by depth limits and minimum leaf sizes; neural networks by dropout, weight decay, data augmentation and early stopping.

2.10 Model Evaluation Metrics

For classification, metrics derive from the confusion matrix of true positives (TP), false positives (FP), true negatives (TN) and false negatives (FN):

$$\text{Precision} = \frac{TP}{TP+FP}, \quad \text{Recall} = \frac{TP}{TP+FN}, \quad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

The ROC curve plots recall against the false-positive rate across thresholds; its area (AUC) is threshold-independent. On imbalanced medical data, recall and F1 matter more than accuracy, because a missed disease (FN) usually costs more than a false alarm.

For regression:

$$\text{MAE} = \tfrac{1}{n}\textstyle\sum |y_i - \hat{y}_i|, \quad \text{RMSE} = \sqrt{\tfrac{1}{n}\sum (y_i - \hat{y}_i)^2}, \quad R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

For clustering without labels, the silhouette score (−1 to 1, higher is better) and Davies–Bouldin index (lower is better) measure separation and compactness. With labels, the adjusted Rand index (ARI) measures agreement with the true classes.

03 Theoretical Foundations of Deep Learning

Deep learning replaces hand-crafted features with layers of learned transformations, which is why it drives the three image projects in this chapter. This section covers the building blocks those projects rely on.

3.1 Introduction to Deep Learning

A deep network is a composition of simple functions, f(x) = f_L(... f_2(f_1(x))). Early layers learn generic patterns (edges, textures); later layers combine them into task-specific concepts (hyphae, nuclei, lesion borders). Three developments made this practical after 2012: large labelled datasets such as ImageNet, GPU computing, and better training techniques (ReLU, batch normalisation, residual connections).

3.2 Artificial Neural Networks

An artificial neuron computes a weighted sum of its inputs and passes it through a non-linear activation:

$$a = \sigma\left(\sum_j w_j x_j + b\right)$$

A multilayer perceptron (MLP) stacks fully connected layers of such neurons. Training uses backpropagation: the chain rule computes the gradient of the loss with respect to every weight, and an optimiser updates the weights a small step against that gradient. MLPs are the classification heads on top of every CNN in Chapters 6–8.

3.3 Convolutional Neural Networks (CNNs)

CNNs exploit two properties of images: nearby pixels are related (locality) and a pattern means the same thing wherever it appears (translation equivariance). A convolution layer slides small learned filters across the image to produce feature maps; pooling layers downsample those maps; fully connected layers at the end make the decision.

Table 3. Key architectures referenced in the case studies

Architecture	Year	Key idea	Parameters (approx.)
VGG16	2014	Deep stacks of 3×3 convolutions	138 M
InceptionV3	2015	Parallel filters of several sizes per block	24 M
ResNet50	2015	Residual (skip) connections enable very deep nets	25.6 M
Xception	2017	Depthwise separable convolutions	22.9 M
DenseNet121	2017	Each layer receives all earlier feature maps	8 M
MobileNetV2	2018	Inverted residuals, built for phones	3.5 M
EfficientNetB0	2019	Compound scaling of depth, width, resolution	5.3 M

3.4 Transfer Learning and Pretrained Models

Transfer learning reuses a network trained on a large source dataset (usually ImageNet's 1.28 million images) as the starting point for a smaller target task. Two strategies are common. Feature extraction freezes the pretrained backbone and trains only a new classification head. Fine-tuning then unfreezes some or all backbone layers and continues training at a low learning rate.

Transfer learning is essential for medical imaging, where labelled datasets hold thousands rather than millions of images. It shortens training, reduces overfitting and usually improves accuracy, even though microscopy and histopathology look nothing like ImageNet photographs.

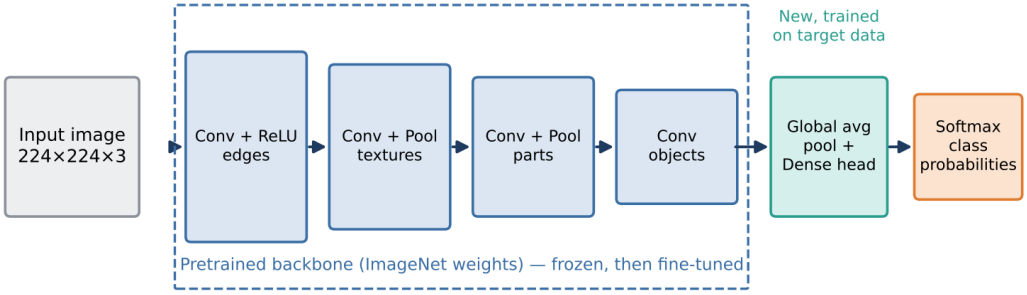


Figure 3. A convolutional neural network used with transfer learning: a pretrained backbone extracts features and a new head is trained on the target classes.

3.5 Attention Mechanisms

Attention lets a model weight the parts of its input that matter for the current decision. Scaled dot-product attention, the core of the Transformer, is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)V$$

The Vision Transformer (ViT) splits an image into patches, embeds each patch as a token and applies self-attention across all patches, which captures long-range relationships a CNN sees only in deep layers. Lighter variants add attention to CNNs: squeeze-and-excitation blocks weight channels, and CBAM weights both channels and spatial positions.

3.6 Recurrent Neural Networks (RNNs) and LSTMs

RNNs process sequences one step at a time and carry a hidden state forward, which suits time series such as daily stock prices. Plain RNNs struggle to remember long histories because gradients vanish over many steps.

The long short-term memory (LSTM) cell fixes this with three gates — input, forget and output — that control what is written to, kept in and read from a memory cell. GRUs simplify the design to two gates. Temporal convolutional networks and Transformers are newer alternatives for sequences.

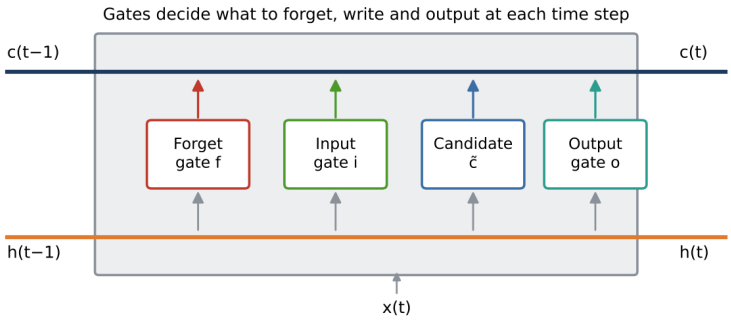


Figure 4. Structure of an LSTM cell, with forget, input and output gates controlling the memory cell.

3.7 Optimization Algorithms and Hyperparameter Tuning

Stochastic gradient descent (SGD) updates weights using mini-batches. Momentum smooths the updates. Adam combines momentum with a per-parameter adaptive learning rate and is the default in most of the projects here.

Hyperparameters (learning rate, batch size, dropout rate, number of trees, regularisation strength) are not learned by gradient descent and must be searched. Grid search tries every combination; random search samples the space and is more efficient in high dimensions; Bayesian optimisation builds a model of the score surface and samples where improvement is most likely. The stock project uses full grid search over both its recurrent networks and its tree models; the PCOS and fungi projects use fixed, literature-based settings with early stopping.

3.8 Loss Functions and Activation Functions

Table 4. Loss Functions and Activation Functions

Function	Type	Formula / behaviour	Typical use
ReLU	Activation	max(0, z)	Hidden layers of CNNs and MLPs
Sigmoid	Activation	1 / (1 + e^(−z))	Binary output
Softmax	Activation	$e^{(z_k)} / \Sigma e^{(z_j)}$	Multi-class output
Binary cross-entropy	Loss	−[y log p + (1−y) log(1−p)]	PCOS yes/no
Categorical cross-entropy	Loss	$-\Sigma y_k \log p_k$	LC25000, fungi, skin classes
Focal loss	Loss	Down-weights easy examples	Class-imbalanced images
Mean squared error	Loss	mean of $(y - \hat{y})^2$	Stock price regression

3.9 Deep Learning Model Evaluation

Deep models use the same metrics as Section 2.10, plus training diagnostics. Learning curves of training and validation loss reveal overfitting (the curves diverge) and underfitting (both plateau high). Per-class confusion matrices show which classes are confused, for example the two lung carcinoma subtypes. Macro-averaged F1 treats every class equally and is the fairer summary when classes are unbalanced.

3.10 Explainable AI and Model Reliability

Explainable AI (XAI) makes model decisions inspectable. For images, Grad-CAM highlights the regions that drove a CNN's prediction, so a pathologist can check whether the model looked at tumour cells or at a slide artefact. For tabular data, SHAP assigns each feature a contribution to each prediction based on Shapley values; LIME fits a simple local surrogate model around one prediction.

Reliability also means knowing when the model is unsure. Calibration checks whether a predicted probability of 0.9 is correct about 90% of the time. Monte Carlo dropout and deep ensembles estimate prediction uncertainty, so low-confidence cases can be routed to a human expert.

04 Data Preparation and Experimental Methodology

All five projects follow the same seven-stage workflow: collect, clean, explore, split, transform, train and tune, then evaluate on held-out data. The single rule that ties the stages together is that nothing learned from the test set may influence any earlier stage.

1. Data collection and description
2. Cleaning, missing values and outliers
3. Exploratory data analysis
4. Train–validation–test split (stratified, grouped or chronological)
5. Transformations fitted on training data only (scaling, encoding, augmentation, resampling)

6. Model training and hyperparameter optimisation on training and validation data
7. Single final evaluation on the test set, with confidence intervals where possible

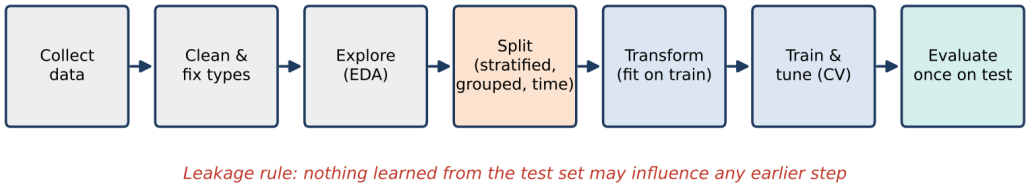


Figure 5. Common experimental workflow followed by all five projects.

4.1 Data Collection and Dataset Description

Table 5. Data Collection and Dataset Description

Project	Source	Size	Target
PCOS	Kaggle clinical dataset (Kerala, India) + lifestyle survey	541 × 45 clinical; 173 × 36 survey	PCOS yes / no
Fungi	DeFungi direct-microscopy dataset	Five hyphal-pattern classes (H1, H2, H3, H5, H6)	Morphotype
Skin disease	Dermatology image dataset	Multiple disease classes	Disease category
Colon & lung	LC25000 histopathology	25,000 images, 5 classes	Tissue class
Google stock	Historical daily prices of GOOGL	Daily OHLCV rows	Closing price

A dataset description should always record: who collected the data and how, the number of samples and features, the label definition, the class balance, the licence, and known biases (single hospital, single scanner, single country).

4.2 Data Cleaning and Preprocessing

Real datasets contain errors that silently break models. The PCOS clinical file, for example, holds a literal letter "a" in the AMH column and the value "1.99." in II beta-HCG; either one makes pandas read the whole column as text. Other defects found in the projects include a near-empty column ("Unnamed: 44", 539 of 541 values missing), a height column labelled in feet that actually holds centimetres, and an ambiguous target value that had to be dropped.

Cleaning steps, in order: fix data types, remove duplicate rows, correct unit errors, drop identifier columns (serial number, patient file number), and drop any column that encodes the answer (for PCOS, "PCOS_Medication" is target leakage).

4.3 Missing Values and Outlier Handling

Missing values are handled by deletion (when a column is mostly empty), simple imputation (median for skewed numeric data, mode for categories), or model-based imputation (k-NN, MICE). The imputer must be fitted on the training split only.

Outliers are detected with the interquartile-range rule (below Q1 − 1.5·IQR or above Q3 + 1.5·IQR), z-scores, or Isolation Forest. In clinical data, an extreme value may be a real patient, so outliers are inspected before removal rather than dropped automatically.

4.4 Image Preprocessing and Augmentation

Images are resized to the backbone's input size (for example 224×224 for ResNet and EfficientNet, 299×299 for Xception and InceptionV3) and normalised with the ImageNet channel mean and standard deviation. Microscopy and histopathology images may also need stain or colour normalisation, because staining varies between labs and batches.

Augmentation creates plausible new training images: rotations, horizontal and vertical flips (valid for slides, which have no "up"), small zooms and shifts, brightness and contrast changes. Augmentation is applied to training images only; validation and test images are only resized and normalised.

4.5 Feature Scaling and Encoding

Distance- and gradient-based models (k-NN, SVM, logistic regression, neural networks, K-means, PCA) need features on comparable scales. Standardisation gives each feature mean 0 and standard deviation 1; min–max scaling maps it to [0, 1]. Tree models do not need scaling.

Categorical variables are one-hot encoded when unordered (blood group) and ordinal-encoded when ordered (exercise frequency). Yes/no fields become 0/1.

4.6 Exploratory Data Analysis

EDA checks distributions (histograms, box plots), relationships (correlation heatmaps, pair plots), class balance (bar charts) and, for time series, trend, seasonality and volatility. Its purpose is to form hypotheses and catch problems early, not to select features using the test set.

4.7 Train-Test Splitting and Data Leakage Prevention

Data leakage occurs when information from outside the training data enters the model, producing scores that will not hold in practice. Four forms appeared across these projects:

Table 6. Train-Test Splitting and Data Leakage Prevention

Leakage type	Example	Prevention
Preprocessing leakage	Scaler or SMOTE fitted before the split	Put all transforms inside a scikit-learn Pipeline fitted per fold
Group leakage	Overlapping patches from one fungal slide in both train and test	Split by slide or patient (GroupKFold)
Temporal leakage	Random split of stock prices lets the model see the future	Chronological split; TimeSeriesSplit for CV
Target leakage	A feature recorded after diagnosis (PCOS medication)	Drop features unavailable at prediction time

Stratified splits preserve class ratios and matter for small, imbalanced medical sets. Typical ratios are 70:15:15 or 80:10:10 for images and 80:20 with internal k-fold CV for tabular data.



Figure 6. Three ways to split data. Grouped splits keep all images from one patient or slide together; chronological splits keep the test period after the training period.

4.8 Experimental Setup and Software Tools

Table 7. Experimental Setup and Software Tools

Layer	Tools used across the projects
Language and notebooks	Python 3, Jupyter, Google Colab
Data handling	NumPy, pandas
Classical ML	scikit-learn, XGBoost, LightGBM, imbalanced-learn
Deep learning	TensorFlow / Keras, PyTorch, timm
Synthetic data	SDV (CTGAN, TVAE, Gaussian copula)
Visualisation	Matplotlib, Seaborn, Plotly
Explainability	SHAP, LIME, Grad-CAM
Finance data	yfinance or downloaded CSV
Mobile deployment	TensorFlow Lite, Flutter / Android
Hardware	Colab GPU (T4) for image models; CPU for tabular and time-series models

4.9 Model Training and Hyperparameter Optimization

Image models are trained in two phases: a few epochs with the backbone frozen, then fine-tuning with a learning rate 10–100 times smaller. Callbacks stop training when validation loss stops improving (early stopping), reduce the learning rate on a plateau, and save the best checkpoint.

Tabular models are tuned with grid, random, halving or Bayesian search inside cross-validation. Class imbalance is handled with class weights, threshold tuning, or resampling (SMOTE, ADASYN) applied inside each training fold only.

4.10 Reproducibility and Experimental Documentation

A result is only useful if someone else can reproduce it. Each project should fix random seeds (Python, NumPy, framework), record library versions, save the exact train/validation/test file lists, store trained weights, and publish code and results in a version-controlled repository.

The repositories behind this chapter follow that practice to varying degrees: the PCOS repository publishes notebooks plus result CSVs and every figure, and the colon–lung repository publishes the architecture, results and paper. Section 13.3 lists them all.

CASE STUDY I

05 Machine Learning for PCOS Data Analysis

KEY RESULT

On two separate PCOS cohorts, synthetic oversampling gave no significant gain on the clinical data (best F1 0.835 vs 0.833 baseline) and a small, non-significant gain on the survey data (TabDDPM-lite F1 0.732 vs 0.686). Unsupervised methods found real structure only in the clinical cohort, where K-Means reached 0.810 held-out balanced accuracy and Isolation Forest 0.890 ROC-AUC. Code and results: Implementation-in-PCOS.

5.1 Introduction to Polycystic Ovary Syndrome (PCOS)

PCOS is the most common endocrine disorder in women of reproductive age, affecting roughly 6–13% worldwide, and up to 70% of cases go undiagnosed (WHO). Diagnosis normally follows the Rotterdam criteria: at least two of oligo- or anovulation, clinical or biochemical hyperandrogenism, and polycystic ovaries on ultrasound. PCOS raises the long-term risk of type 2 diabetes, cardiovascular disease and infertility.

- Problem: diagnosis needs ultrasound and hormone tests that are costly and unevenly available; small, imbalanced datasets make ML models unstable.
- Research gap: most published PCOS-ML papers use the same 541-row Kaggle dataset, report very high accuracy from single splits, rarely test synthetic data rigorously, and seldom explore unsupervised structure.
- Objectives: (1) benchmark eight synthetic-data generators for oversampling on utility, fidelity and privacy; (2) test whether PCOS emerges as clusters or anomalies without labels; (3) explain both with SHAP and LIME.

5.2 Clinical and Survey Dataset Description

Two files were supplied as "Training data.csv" and "Testing data.csv", but they share zero columns, so they are treated as two independent cohorts rather than a train/test split.

Table 8. Clinical and Survey Dataset Description

Property	Cohort A: clinical	Cohort B: lifestyle survey
Raw shape	541 rows × 45 columns	173 rows × 36 columns
After cleaning	541 rows, 41 features	172 rows, 34–39 features
Target	PCOS (Y/N)	PCOS (Yes/No)
Positives	177 (32.7%)	38 (22.1%)
Feature groups	Anthropometrics, cycle, hormones (FSH, LH, AMH, TSH, PRL, beta-HCG), symptoms, blood pressure, follicle counts and sizes, endometrium	Age band, marital status, family history, menstrual irregularity, hyperandrogenism, hirsutism, insulin resistance, 11 diet items, exercise, sleep, stress, smoking

Univariate Mann–Whitney U tests found 24 of 41 clinical features and 16 of 39 survey features significant at $p < 0.05$. The strongest were right and left follicle counts in the clinical cohort ($p = 3.1 \times 10^{-48}$ and 1.7×10^{-41}) and hyperandrogenism, hirsutism and insulin resistance in the survey cohort.

5.3 Data Preprocessing and Feature Selection

- Clinical: dropped serial number, patient file number and the near-empty "Unnamed: 44" column; coerced all columns to numeric (fixing "a" in AMH and "1.99." in II beta-HCG); median imputation of 4 cells; Blood Group and binary fields treated as categorical.

- Survey: dropped one row with an ambiguous target; dropped PCOS_Medication as target leakage (only PCOS-positive respondents list a medication); ordinal-encoded age, exercise and sleep bands; mapped multi-select answers to 0/1; renamed Height_ft to Height_cm because the values are centimetres, and derived BMI.
- Scaling: StandardScaler inside model pipelines; QuantileTransformer plus one-hot encoding for the diffusion generators.
- Splits: seed 42; stratified 75/25 split repeated with seeds 42, 43 and 44 for the oversampling benchmark; stratified 5-fold CV for unsupervised evaluation, with scalers fitted per fold.
- No separate feature-selection step: all cleaned features were retained so that generators and explainers saw the full feature space.

5.4 Supervised and Unsupervised Learning Approaches

The supervised benchmark (PCOD.ipynb) fits each generator on the real training split only, generates enough synthetic minority rows to balance the classes, adds them to the real training data, and scores three classifiers on the untouched real test split.

Table 9. Supervised and Unsupervised Learning Approaches

Generator	Type	Key settings
SMOTE, ADASYN, Borderline-SMOTE	Interpolation	k = 5 neighbours
CTGAN	GAN	300 epochs; minority rows filtered from a large sample pool
TVAE	Variational autoencoder	300 epochs
TabDDPM-lite	Diffusion (own re-implementation)	3×256 MLP denoiser, cosine schedule, T = 200, 4,000 steps, AdamW lr 1e-3
TabSyn-lite	Latent diffusion (own re-implementation)	Conditional VAE (latent 16) + diffusion in latent space
CTAB-GAN+	GAN	Not run: the code fell back to CTGAN, so its row is effectively a second CTGAN seed

Classifiers: Random Forest (400 trees), Logistic Regression (standardised, max_iter 3,000) and XGBoost (400 trees, depth 4, learning rate 0.08). Utility is averaged across all three classifiers and three repeats.

The unsupervised study (PCOS_Unsupervised.ipynb) runs five clustering methods, six embeddings and three anomaly detectors, each evaluated against the held-out labels in 5-fold CV.

5.5 Clustering and Pattern Discovery

Clustering used K-Means (k = 2, 25 initialisations), agglomerative clustering (Ward, average and complete linkage), DBSCAN, HDBSCAN and Gaussian mixtures. DBSCAN and HDBSCAN collapsed to all-noise in the full standardised space, so the code searched the full space, then PCA-90%, then UMAP-2D, and reports which space was used.

Table 10. Clustering and Pattern Discovery

Method	Cohort A held-out bal. acc.	Cohort B held-out bal. acc.
K-Means	0.810 ± 0.011	0.500 ± 0.000
Hierarchical (Ward)	0.720 ± 0.076	0.505 ± 0.011
DBSCAN	0.631 ± 0.063	0.500 ± 0.000
HDBSCAN	0.617 ± 0.017	0.556 ± 0.124
GMM	0.489 ± 0.026	0.514 ± 0.032

Anomaly detectors were trained only on PCOS-negative training rows, treating PCOS as deviation from the healthy pattern. Isolation Forest was best in both cohorts: ROC-AUC 0.890 ± 0.022 (A) and 0.802 ± 0.076 (B). The

contrast is the key finding: survey PCOS cases do not form a separate cluster, but they do sit away from the healthy majority.

On the full clinical data, K-Means had a low silhouette (0.091) yet an ARI of 0.429 with the true labels. Average and complete linkage produced a high silhouette (0.746) with ARI −0.002, a degenerate split in which almost all points fall into one cluster — a reminder that internal cluster scores alone can mislead.

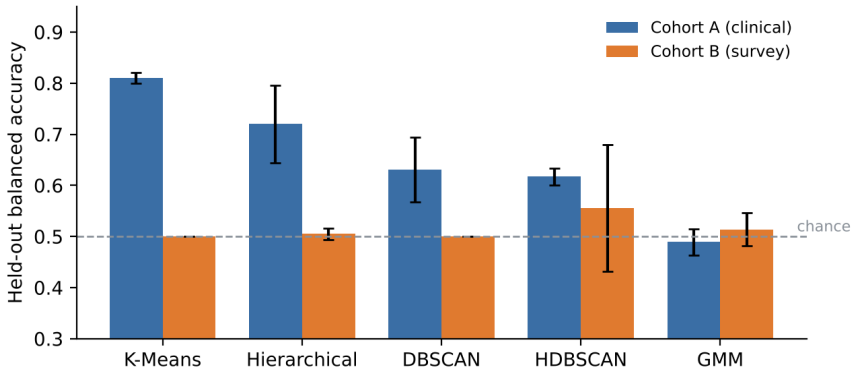


Figure 7. Held-out balanced accuracy of five clustering methods on the two PCOS cohorts (5-fold CV, mean ± SD).

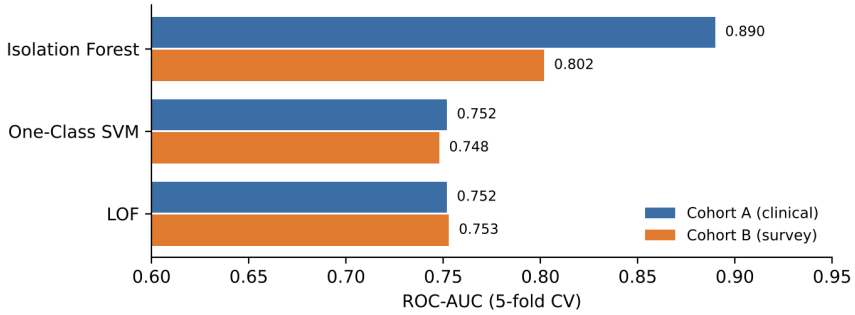


Figure 8. Anomaly-detection ROC-AUC on the two PCOS cohorts, with detectors trained on PCOS-negative cases only.

5.6 Principal Component Analysis (PCA)

Twenty-nine principal components were needed to keep 90% of variance in the clinical cohort and 25 in the survey cohort, showing that information is spread across many weakly correlated features. Each 2-D embedding was scored by 5-NN balanced accuracy on held-out folds:

Table 11. Principal Component Analysis (PCA)

Embedding	Cohort A	Cohort B
LDA (supervised, for reference)	0.901	0.812
UMAP	0.832	0.686
ICA	0.831	0.792
PCA	0.829	0.792
Factor analysis	0.829	0.845
t-SNE	0.827	0.720

In the oversampling study, PCA fitted on real training data was also used to overlay synthetic and real PCOS rows (Figure 10), giving a visual check of whether each generator covers the real minority distribution.

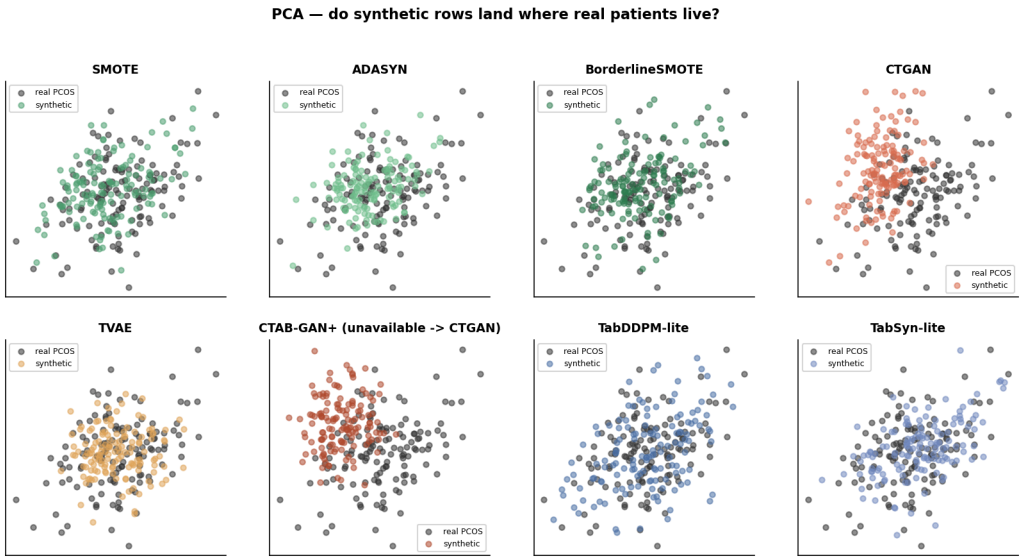


Figure 9. PCA overlap of real PCOS cases and synthetic rows from each generator, clinical cohort. Source: Implementation-in-PCOS repository.

5.7 Comparison of Model Utility and Performance

Table 12. Clinical cohort, mean over 3 classifiers × 3 repeats

Generator	Accuracy	Recall	F1 (± sd)	ROC-AUC	PR-AUC	Composite
None (baseline)	0.899	0.780	0.833 (0.022)	0.946	0.924	0.819
TabSyn-lite	0.889	0.841	0.831 (0.029)	0.944	0.923	0.763
TVAE	0.896	0.813	0.835 (0.018)	0.946	0.923	0.685
TabDDPM-lite	0.881	0.841	0.821 (0.020)	0.945	0.918	0.643
CTGAN	0.891	0.816	0.830 (0.025)	0.941	0.918	0.517
Borderline-SMOTE	0.886	0.826	0.822 (0.035)	0.942	0.911	0.449
ADASYN	0.879	0.823	0.814 (0.024)	0.942	0.912	0.386
SMOTE	0.878	0.813	0.811 (0.028)	0.940	0.910	0.283

Table 13. Survey cohort

Generator	Accuracy	Recall	F1 (± sd)	ROC-AUC	PR-AUC	Composite
TabDDPM-lite	0.881	0.765	0.732 (0.093)	0.897	0.797	0.805
TabSyn-lite	0.884	0.679	0.705 (0.120)	0.877	0.771	0.777
None (baseline)	0.884	0.630	0.686 (0.146)	0.882	0.775	0.620
CTGAN	0.863	0.691	0.676 (0.174)	0.864	0.717	0.465
TVAE	0.845	0.642	0.624 (0.154)	0.866	0.687	0.269
ADASYN	0.822	0.630	0.596 (0.134)	0.834	0.669	0.272
SMOTE	0.829	0.642	0.602 (0.115)	0.845	0.671	0.266
Borderline-SMOTE	0.824	0.605	0.590 (0.115)	0.840	0.676	0.230

The composite score weights utility 60% (F1, ROC-AUC, PR-AUC), fidelity 30% and privacy 10%, after min–max normalisation within each run. Diffusion-based generators raised recall most: on the clinical data, TabSyn-lite and TabDDPM-lite lifted recall from 0.780 to 0.841 at a small cost in precision. Interpolation methods (SMOTE family) lowered F1 on both cohorts.

Runtime ranged from near zero for the SMOTE family to 16.8 s for TVAE and 41.8–53.2 s for the GAN and diffusion generators on the clinical data (CPU).

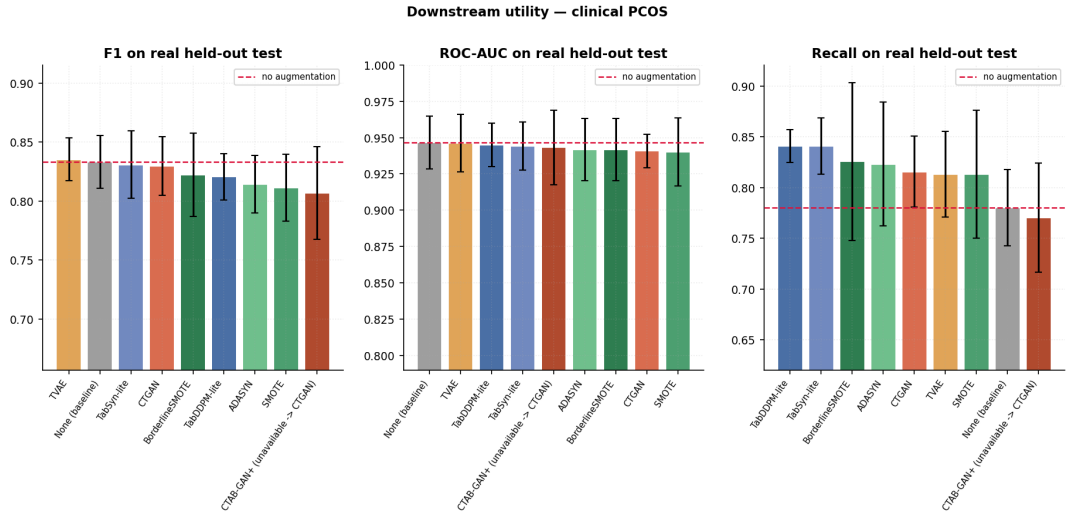


Figure 10. Downstream utility (F1, ROC-AUC, recall) on the real held-out test set for each generator, clinical cohort. Source: Implementation-in-PCOS repository.

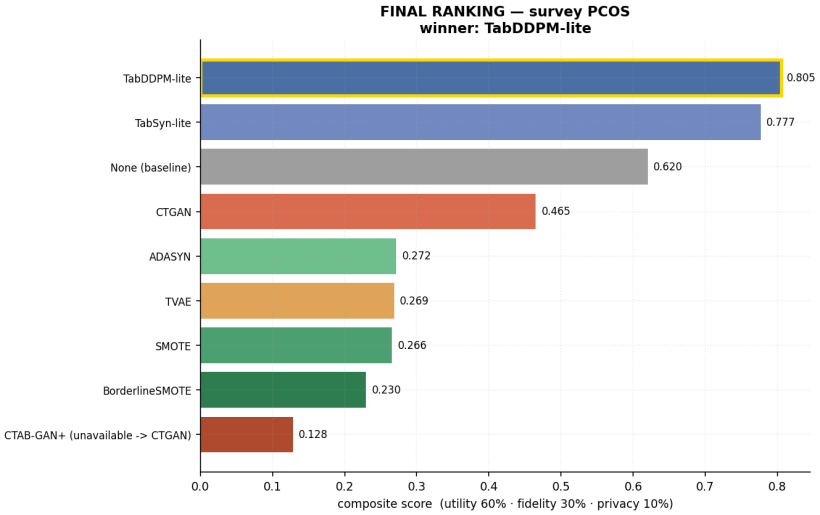


Figure 11. Composite ranking of generators on the survey cohort (utility 60%, fidelity 30%, privacy 10%). Source: Implementation-in-PCOS repository.

5.8 ROC Curves and Classification Metrics

ROC curves per classifier and generator (Figure 13) show that all methods sit within a narrow band on the clinical data (ROC-AUC 0.940–0.946), so ranking differences come mostly from the threshold-dependent metrics. On the survey data, the spread is wider (0.834–0.897) and the F1 standard deviations (0.09–0.17) are large because each test split holds only 9 positive cases.

PR-AUC is reported alongside ROC-AUC because, with 22–33% positives, precision–recall curves are more sensitive to minority-class performance.

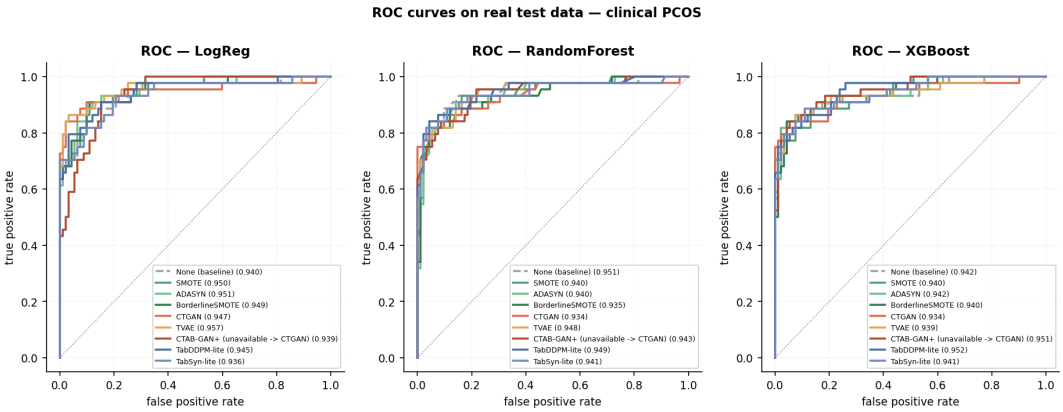


Figure 12. ROC curves for logistic regression, random forest and XGBoost trained with each generator, clinical cohort. Source: Implementation-in-PCOS repository.

5.9 Privacy, Synthetic Data Fidelity and Reliability

Table 14. Privacy, Synthetic Data Fidelity and Reliability

Measure	What it computes	Better
KS mean	Mean Kolmogorov–Smirnov statistic between real and synthetic numeric columns	Lower
TVD mean	Mean total variation distance over categorical columns	Lower
CorrDiff	Mean absolute difference between real and synthetic correlation matrices	Lower
DCR median	Median distance from each synthetic row to its nearest real training row (standardised)	Higher (less copying)

TabSyn-lite produced the most faithful data (clinical KS 0.136, TVD 0.051, CorrDiff 0.085) but, on the survey cohort, also the lowest privacy distance (DCR 2.298). CTGAN showed the opposite: the largest DCR (6.83 clinical) with the worst KS (0.343). This fidelity–privacy trade-off is expected: the closer synthetic rows track real ones, the more they risk resembling individual patients. No generator fell below the 0.5 memorisation-risk line used in Figure 14.

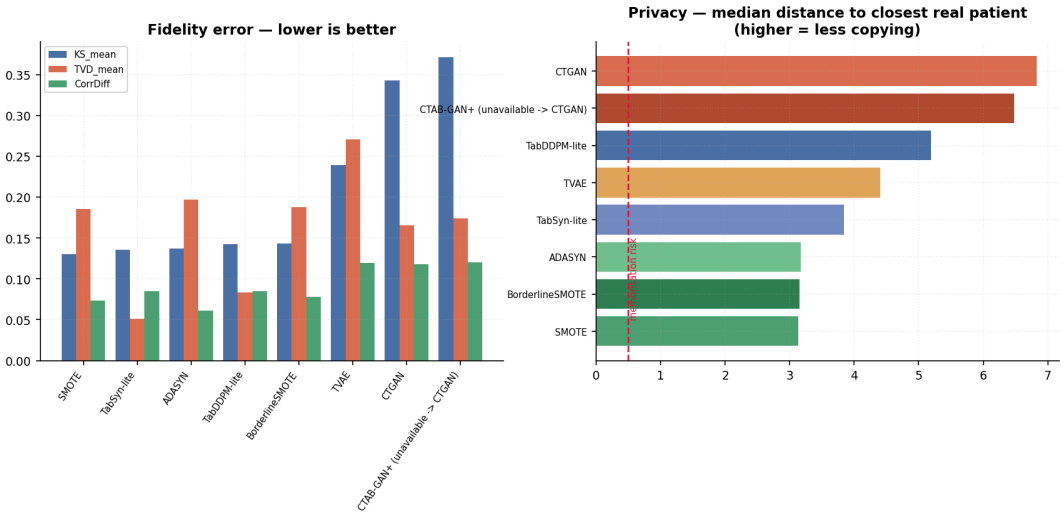


Figure 13. Fidelity errors (left, lower is better) and privacy distance to the closest real record (right, higher is better), clinical cohort. Source: Implementation-in-PCOS repository.

5.10 Interpretation of Experimental Results

- Synthetic oversampling is not a free win. On the larger clinical cohort, no generator beat the unaugmented baseline by more than its own standard deviation. On the smaller survey cohort, diffusion models helped most but still within noise.

- Clinical PCOS is a separable phenotype: K-Means recovered it without labels (0.810 balanced accuracy), and SHAP ranked skin darkening, cycle regularity, weight gain, fast food, hair growth and right follicle count highest — closely matching the Rotterdam criteria.
- Survey PCOS is not a cluster but a deviation: clustering fell to chance while Isolation Forest still reached 0.80 ROC-AUC. Its top SHAP features were diet items (fats, starchy vegetables, sweets), insulin resistance and hyperandrogenism.
- SHAP and LIME agreed moderately (Spearman ρ 0.593 clinical, 0.397 survey; 6 of top-10 features shared), so explanations should be cross-checked rather than trusted from one method.

Limitations: single-source cohorts with no external validation; median imputation in the oversampling notebook was fitted before the split (a minor leakage affecting 4 cells); CTAB-GAN+ was not actually run; TabDDPM and TabSyn are compact re-implementations; small survey test splits make F1 unstable.

Conclusion: for small tabular medical datasets, report repeated-split variance, test synthetic data against an unaugmented baseline, and check unsupervised structure — it can reveal whether a condition is a distinct group or a departure from normal.

06 CASE STUDY II
Deep Learning for Fungal Image Classification

KEY RESULT

On 902 held-out DeFungi microscopy patches, a fine-tuned ConvNeXtV2-Tiny reached 85.1% accuracy (macro-F1 0.883), while the proposed morphology-guided fusion model, MorphoFuse, reached 75.4% — a negative result reported honestly. The study's main contribution is methodological: a leakage audit, calibration, conformal prediction and taxonomy-aware error analysis that none of the prior DeFungi studies report. Implementation: Fung-implementation.

6.1 Introduction to Fungal Microscopy and Image Classification

Superficial fungal infections of skin, hair and nails affect an estimated 20–25% of the world's population. Direct microscopic examination of a potassium-hydroxide preparation is the fastest first test, but it depends on an experienced mycologist recognising hyphal patterns by eye. Telling dermatophytes from non-dermatophyte moulds matters because the two need different treatments.

- Problem: automate classification of hyphal morphotypes in direct-microscopy images.
- Research gap: prior DeFungi studies report 85–98% accuracy from single splits, with no leakage checks, no calibration, and no test of whether mycological morphology adds information beyond what a CNN sees.
- Objectives: (1) audit the dataset for near-duplicate leakage; (2) benchmark strong CNN and foundation-model backbones under one protocol; (3) test whether fusing 24 morphometric descriptors with visual features improves classification; (4) assess calibration, uncertainty and clinically meaningful errors.

6.2 Dataset Collection and Class Distribution

DeFungi (Pineda Sopo et al., 2021) was built from 660 raw images taken in a Colombian mycology laboratory, cut into 500 × 500-pixel patches. This study uses the Kaggle repack "Microscopic Fungi Images", which distributes the patches in fixed train, validation and test folders.

Table 15. Dataset Collection and Class Distribution

Class	Morphotype	Train	Val.	Test	Total	Share
H1 – TSH	Tortuous septate hyaline hyphae	1,000	437	437	1,874	27.6%
H2 – BASH	Beaded arthroconidial septate hyaline hyphae	1,000	232	233	1,465	21.5%
H3 – GMA	Groups or mosaics of arthroconidia	1,001	81	82	1,164	17.1%
H5 – SHC	Septate hyaline hyphae with chlamydioconidia	1,000	80	80	1,160	17.1%
H6 – BBH	Broad brown hyphae	1,000	69	70	1,139	16.7%
Total		5,001	899	902	6,802	100%

The training folder is balanced but the test folder is not (TSH 48.5%, BBH 7.8%), which implies the distributor resampled the training data. Macro-averaged metrics are therefore essential.

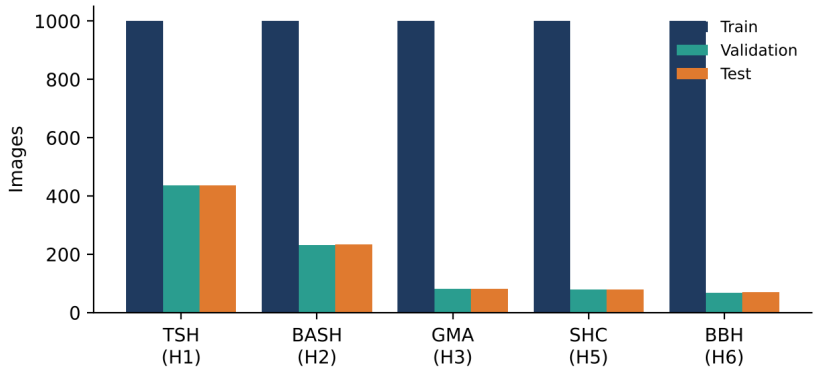


Figure 14. Class distribution of the DeFungi Kaggle repack across training, validation and test folders.

6.3 Microscopic Image Preprocessing

- Resizing: images resized to 256 × 256 and cached; training used random-resized 224 × 224 crops (area 0.55–1.0), evaluation a plain resize to 224.
- Augmentation: 90° rotations and flips (all eight orientations, since microscope fields have no "up"), brightness, contrast and saturation in [0.8, 1.2], and a light blur with probability 0.2.
- Normalisation: ImageNet mean and standard deviation; no stain normalisation.
- Segmentation for morphometrics: CLAHE, Sato tubeness filtering at three scales, Otsu thresholding against a Gaussian background, small-component removal and skeletonisation.

6.4 Morphological Feature Extraction

Table 16. Twenty-four handcrafted descriptors encode what a mycologist looks for

Group	Descriptors
Structure	Foreground area fraction, skeleton density, number of components, branch density, endpoint density
Width	Mean, standard deviation and 90th percentile of hyphal width; beading index
Shape	Tortuosity, round-cell density, mosaic fraction, circularity, fractal dimension, tubularity
Colour	Melanin index, brownness (CIELAB b*), foreground saturation, foreground–background contrast
Texture	GLCM contrast, homogeneity and energy; entropy; orientation coherence

All 24 differed across classes (Kruskal–Wallis, FDR < 10^{-25}). Effect sizes were largest for foreground saturation (ε^2 = 0.457), melanin index (0.330) and tubularity (0.261). A gradient-boosting classifier on morphometrics alone reached 69.7% accuracy and macro-F1 0.714; permutation importance showed it relied mainly on colour (removing melanin index cut macro-F1 by about 0.25), while structural and width descriptors contributed less than 0.01 each.

6.5 CNN-Based and Transformer-Based Feature Extraction

Table 17. CNN-Based and Transformer-Based Feature Extraction

Backbone	Type	Pre-training	Parameters
ResNet50	CNN with residual blocks	ImageNet-1k	23.5 M
ConvNeXtV2-Tiny	Modernised CNN	Masked autoencoder, ImageNet-22k → 1k	27.9 M
DINOv2 ViT-S/14	Vision Transformer	Self-supervised, LVD-142M	21.6 M

DINOv2 provides both a global image token and 16 × 16 patch tokens, which MorphoFuse uses for cross-attention.

6.6 Transfer Learning Using Pretrained Models

All backbones were fully fine-tuned with discriminative learning rates: 1e-3 for new heads, 2e-4 for CNN backbones and 5e-5 for transformer backbones, with half an epoch of warm-up and cosine decay. AdamW (weight decay 0.05), label smoothing 0.1, an exponential moving average of weights (0.99), mixed precision, gradient clipping and stochastic depth 0.1 stabilised training. A frozen DINOv2 backbone with a logistic-regression or 1-NN probe served as a cheap reference.

6.7 Feature Fusion and Classification

MorphoFuse-XAttn (22.5 M parameters) fuses the two information sources in four steps:

1. Project the DINOv2 global token g and patch tokens T to 256 dimensions; encode the 24 morphometrics with an MLP (24 → 128 → 256) into m.
2. Cross-attention: the morphology vector queries the patch tokens, c = MultiHeadAttention(Q = m, K = V = T), with 4 heads, so morphology steers where the model looks.
3. Gated fusion: $\gamma = \sigma(W[g; c; m] + b)$ and $z = \gamma \odot g + (1 - \gamma) \odot c$, letting the network decide per sample how much to trust the morphology-guided view.
4. Classify [z; m] with a two-layer head (dropout 0.3 and 0.2). During training the whole morphometric vector is zeroed with probability 0.15 so the model cannot depend on it entirely.

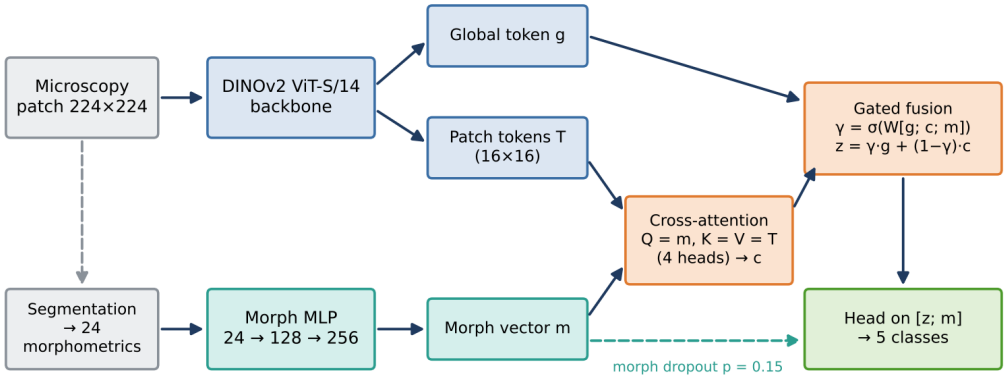


Figure 15. Architecture of MorphoFuse-XAttn: morphometric features query DINOv2 patch tokens through cross-attention, and a learned gate fuses the result with the global token.

6.8 Model Training and Evaluation

All models used batch size 64, up to 10 epochs with early stopping on validation macro-F1 (patience 3), seed 42, on a single NVIDIA T4 GPU in Colab (PyTorch 2.11, timm). Training took 5.4–12.4 minutes per model. Test predictions used eight-orientation test-time augmentation and temperature scaling fitted on half of the validation set.

Table 18. Model Training and Evaluation

Model	Accuracy [95% CI]	Balanced acc.	Macro-F1 [95% CI]	Cohen's κ	Macro-AUC	ECE
Ensemble (top 3)	0.855	0.893	0.887	0.786	0.971	0.048
ConvNeXtV2-Tiny	0.851 [0.828, 0.875]	0.894	0.883 [0.863, 0.903]	0.783	0.975	0.028
ResNet50	0.835 [0.809, 0.858]	0.873	0.867 [0.845, 0.888]	0.757	0.965	0.015
DINOv2 ViT-S/14	0.786 [0.763, 0.812]	0.843	0.825 [0.802, 0.849]	0.689	0.949	0.031
MorphoFuse-XAttn	0.754 [0.727, 0.779]	0.812	0.784 [0.755, 0.810]	0.642	0.934	0.028

McNemar tests confirmed that MorphoFuse was significantly worse than each baseline ($p < 0.001$ vs ResNet50 and ConvNeXtV2; $p = 0.013$ vs DINOv2). Per-class recall shows where it lost: BASH recall fell to 0.537, against 0.71–0.75 for the baselines, with 29% of BASH patches sent to TSH.

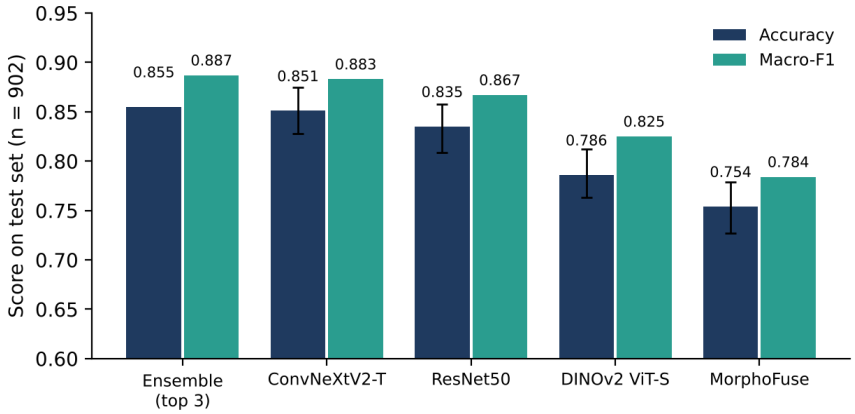


Figure 16. Test accuracy (with 95% bootstrap CI) and macro-F1 of the fungal classifiers.

6.9 Handling Class Imbalance and Data Leakage

The test set's imbalance was handled by reporting balanced accuracy and macro-F1, and by logit adjustment in the loss (inactive here because training data were balanced).

Leakage was the more serious issue. No exact duplicates crossed the official splits, but 1.0–1.1% of validation and test images were near-duplicates of training images (perceptual-hash distance ≤ 4), and 43.5% had a DINOv2 cosine similarity above 0.95 to some training image — adjacent fields of view from the same slide. To estimate the effect, images were clustered into 1,743 pseudo-source groups and evaluated with 5-fold StratifiedGroupKFold:

Table 19. Handling Class Imbalance and Data Leakage

Frozen DINOv2 probe	Official split	Group 5-fold CV (mean ± SD)	Optimism gap
Logistic regression accuracy	0.727	0.748 ± 0.016	−0.021
1-NN accuracy	0.808	0.689 ± 0.021	+0.120

The 1-NN probe lost 12 percentage points once similar images were kept together, showing that memorisation-style models are flattered by slide-level leakage. Because 1,743 pseudo-groups approximate only 660 true source images, the real gap is likely larger. Prior studies reporting 94–98% on random splits may be similarly inflated.

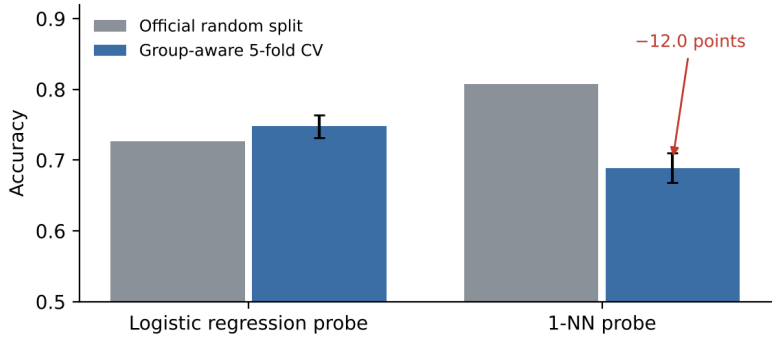


Figure 17. Effect of slide-level leakage: accuracy of frozen DINOv2 probes on the official split versus group-aware cross-validation.

6.10 Model Interpretability and Prediction Uncertainty

- Grad-CAM: ConvNeXtV2 focused on compact hyphal structures; MorphoFuse maps were diffuse and often covered background.

- Fusion gate: median gate values ranged from about 0.51 (BBH) to 0.65 (SHC), so the model leaned on morphology most where colour descriptors are informative — and performed well there (BBH F1 0.95).
- Calibration: temperature scaling cut expected calibration error by 46–78% (ResNet50 from 0.066 to 0.015).
- Conformal prediction at 95% target coverage (LAC method) achieved 0.946 coverage for ConvNeXtV2 with a mean set size of 1.25, and 75.8% of predictions were single-label. Cases with larger sets can be routed to a mycologist.
- Bioinformatics-style profiling: volcano plots gave each morphotype a signature (BBH: high melanin index and saturation, $r \approx 0.78$; SHC: high branch density, $r \approx 0.40$; BASH: only lower saturation, $r \approx -0.37$), mirroring why BASH was hardest for every model.

6.11 Results, Limitations and Clinical Relevance

Table 20. Compared with prior DeFungi work

Study	Model	Reported accuracy	Leakage check
Pineda Sopo et al. (2021)	VGG16 transfer learning, 10-fold	85.0%	No
Cinar & Taspinar (2023)	CNN transfer learning	up to 97.2%	No
Gümüş (2024)	Swin Transformer	98.4%	No
Asadi Amiri & Mohammady (2024)	VGG19 + GAP	97%	No
Sajjadi et al. (2025)	Few-shot graph network	93.1%	No
Fauzi & Suciati (2025)	CBAM-VGG16	94.3%	No
This study	ConvNeXtV2-Tiny, calibrated	85.1% (macro-F1 0.883)	Yes

Clinically, the most important error is confusing a dermatophyte pattern (TSH, BASH, GMA) with a non-dermatophyte mould (SHC, BBH). ConvNeXtV2 made only 10 such cross-group errors in 902 images (group-level accuracy 98.9%), compared with 27 for MorphoFuse; most errors were within the dermatophyte group, where treatment is the same.

Limitations: one dataset release from a single laboratory; patch-level rather than patient-level evaluation; single seed and a 10-epoch budget, with transformers still improving; group-aware fine-tuning and a concatenation-fusion ablation implemented but not yet run; morphometrics measured on down-sampled 256-pixel images.

Conclusion: strong modern CNNs already capture what handcrafted morphometrics describe, so simple fusion added noise rather than signal on DeFungi. The lasting lesson is about evaluation: leakage-aware splits, calibration and clinically grouped errors give a far more honest picture than a single accuracy figure. External validation on DIFaS and SFMID is the planned next step.

07 CASE STUDY III
Deep Learning for Skin Disease Detection

KEY RESULT

This case study takes a trained skin-image classifier out of the notebook and onto a phone, so that a user can photograph a lesion and receive a screening suggestion in seconds. The case study concentrates on system design — data pipeline, model choice, conversion for mobile and application architecture. Implementation: skin-disease-detection-mobile-app.

7.1 Introduction to Automated Skin Disease Detection

Skin conditions are among the most common reasons people seek medical care, and access to dermatologists is limited in rural Sri Lanka and much of South Asia. Esteva et al. (2017) showed that a CNN trained on about 129,000 clinical images could classify skin cancer at dermatologist level. Since then, smartphone cameras and on-device inference have made screening apps practical.

Problem statement: build a mobile application that classifies a photograph of a skin lesion into one of several disease categories and returns the result with a confidence score, without sending the image to a server.

7.2 Skin Image Datasets and Disease Categories

Table 21. Public datasets commonly used for this task

Dataset	Images	Classes	Notes
HAM10000 (ISIC 2018)	10,015 dermoscopic	7 (melanoma, nevus, BCC, AK, BKL, DF, vascular)	Strongly imbalanced; nevus \~67%
ISIC 2019	25,331 dermoscopic	8	Superset of HAM10000
DermNet	\~19,500 clinical	23 categories	Phone-like photos, includes eczema, psoriasis, acne

Clinical (phone-style) images such as DermNet match the deployment setting best, while dermoscopic sets such as HAM10000 offer cleaner labels; combining both improves robustness.

7.3 Image Resizing, Normalization and Augmentation

Images are resized to the backbone's input size (typically 224×224), scaled to [0, 1] or normalised with ImageNet statistics, and augmented during training with rotation, flips, zoom, and brightness and contrast shifts. Brightness and colour augmentation matter more here than for slides, because phone photos vary in lighting, skin tone and camera white balance. Hair removal (DullRazor-style morphological filtering) is optional for dermoscopic images.

The same resize and normalisation must be reproduced exactly inside the app; a mismatch between training and on-device preprocessing is the most common cause of a model that works in the notebook but fails on the phone.

7.4 CNN Architectures for Skin Image Classification

Lightweight backbones are preferred because the model must run on a phone. MobileNetV2 (3.5 M parameters) and EfficientNet-B0 (5.3 M) give a good accuracy-to-size ratio; ResNet50 and DenseNet121 are stronger but heavier.

7.5 Transfer Learning and Fine-Tuning

The backbone is initialised with ImageNet weights and trained in two phases (Section 4.9): head-only training with the backbone frozen, then fine-tuning of the top blocks at a low learning rate (for example 1e-5 with Adam). Class weights or focal loss compensate for imbalance such as HAM10000's dominance of benign nevi.

7.6 Attention-Based Image Classification

Squeeze-and-excitation or CBAM attention blocks can be added to the CNN to emphasise lesion regions over surrounding skin, and Vision Transformers are an alternative when compute allows. For a mobile model, attention is usually limited to light channel attention, which EfficientNet already contains.

7.7 Model Performance Evaluation

Report accuracy, macro precision, recall and F1, per-class recall (especially for melanoma, where a miss is dangerous), a confusion matrix and ROC-AUC per class. Also report model size (MB) and on-device latency (ms per image), since a mobile model is judged on both.

Table 22. Model Performance Evaluation

Metric	Why it matters for a mobile screening app
Test accuracy and macro-F1	Overall and class-balanced correctness
Recall of the most critical class (e.g. melanoma)	A missed malignancy is the costliest error
Per-class confusion matrix	Shows which look-alike conditions are confused
Model size after conversion (MB)	Determines app download size
On-device inference time (ms)	Determines whether results feel instant
Accuracy drop after quantisation	Confirms the converted model still performs

7.8 Integrating the Trained Model into a Mobile Application

The trained Keras or PyTorch model is exported and converted to a mobile format: TensorFlow Lite (`.tflite`) for Android and Flutter, or Core ML for iOS. Post-training quantisation (float16 or int8) cuts the model size by two to four times with a small accuracy loss, which must be measured on the test set after conversion.

1. Train and validate the model in Python.
2. Convert to TensorFlow Lite, with optional quantisation.
3. Re-evaluate the converted model on the test set.
4. Bundle the `.tflite` file and a labels file in the app's assets.
5. In the app, load the interpreter, preprocess the camera image exactly as in training, run inference, and map the output vector to class names.

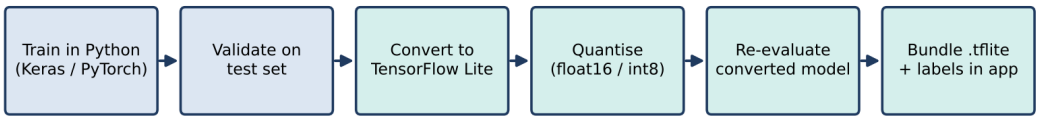


Figure 18. Pipeline for moving a trained skin-disease model onto a phone with TensorFlow Lite.

7.9 Mobile Application Architecture and User Interaction

The app follows a layered design. The presentation layer holds the camera or gallery screen, a preview, and a result screen showing the predicted class, a confidence bar and the top-3 alternatives. The logic layer handles

image preprocessing and calls the inference service. The model layer wraps the TensorFlow Lite interpreter. An optional data layer stores scan history locally.

User flow: open app → capture or choose a photo → crop to the lesion → run on-device inference → view result with confidence and a "see a doctor" message → optionally save to history.

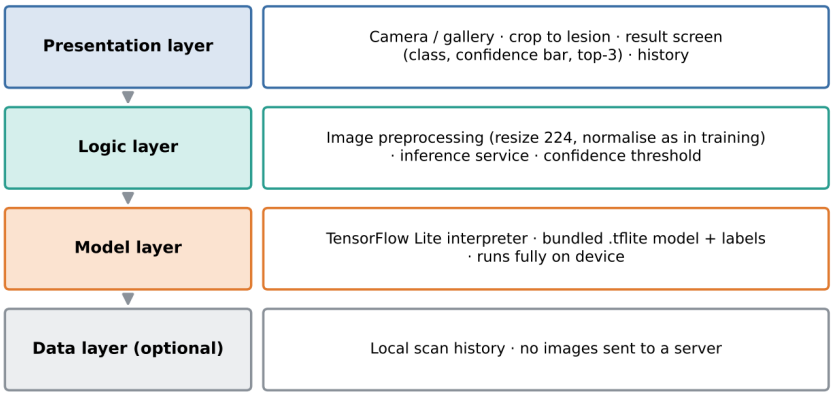


Figure 19. Layered architecture of the skin disease detection mobile application.

7.10 Privacy, Security and Responsible Medical AI

On-device inference means photographs never leave the phone, which removes most data-protection risk. The app should show a clear disclaimer that it is a screening aid and not a diagnosis, display confidence honestly, and advise professional review for any suspicious result. Performance on darker skin tones must be checked, because most public dermatology datasets under-represent them.

7.11 Limitations and Future Improvements

- Public dermoscopy images differ from ordinary phone photos, so accuracy can drop in real use.
- Class imbalance and skin-tone imbalance in training data limit fairness.
- No clinical validation with dermatologists yet.
- Future work: collect locally sourced images with consent, add Grad-CAM overlays in the app, add an "unknown / low confidence" output, and run a pilot study with clinicians.

CASE STUDY IV

08 Deep Learning for Colon and Lung Cancer Image Classification

KEY RESULT

Eight deep models were compared on the five-class LC25000 histopathology dataset; Xception scored 1.000 on every test metric, followed by ResNet50 (0.994 test accuracy) and a custom CNN (0.983). The perfect score needs careful interpretation because LC25000 is itself built by augmenting a small number of original images (Section 8.11). Source: Colon-lung-cancer-image-data, which holds the SICET 2025 paper, architecture notes and result tables.

8.1 Introduction to Cancer Histopathology Image Analysis

Lung and colorectal cancers are the first and second leading causes of cancer death worldwide (GLOBOCAN 2022). Diagnosis is confirmed by a pathologist examining haematoxylin-and-eosin (H&E) stained tissue under a microscope, a slow task that depends on expert availability and suffers from inter-observer variation.

- Problem: automate the distinction between benign tissue and adenocarcinoma or squamous cell carcinoma in lung and colon slides.
- Research gap: prior LC25000 studies often test one or two architectures, use different preprocessing, and rarely report training time alongside accuracy.
- Objectives: compare eight CNN and transformer models under a common protocol with extensive preprocessing, and report accuracy, per-class metrics and training cost.

8.2 Colon and Lung Cancer Image Dataset

LC25000 (Borkowski et al., 2020; distributed on Kaggle) contains 25,000 colour images of 768 × 768 pixels in five balanced classes of 5,000 images each. The images were generated from 750 lung and 500 colon original tissue images by augmentation (rotations and flips).

Table 23. Colon and Lung Cancer Image Dataset

Class	Organ	Label	Images
Lung adenocarcinoma	Lung	lung_aca	5,000
Lung squamous cell carcinoma	Lung	lung_scc	5,000
Lung benign tissue	Lung	lung_n	5,000
Colon adenocarcinoma	Colon	colon_aca	5,000
Colon benign tissue	Colon	colon_n	5,000

The data were split 80:10:10 into training, validation and test sets, stratified by class, giving 500 test images per class.

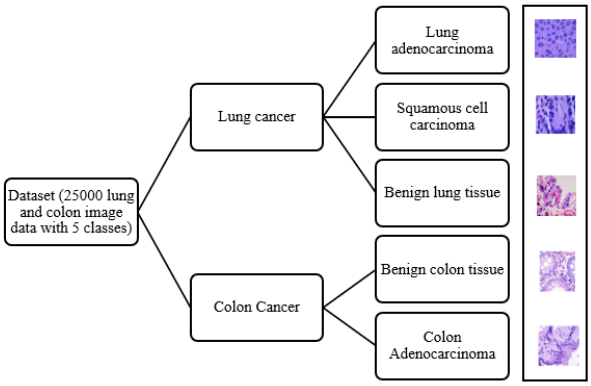


Figure 20. LC25000 dataset structure: 25,000 lung and colon images in five classes. Source: SICET 2025 paper.

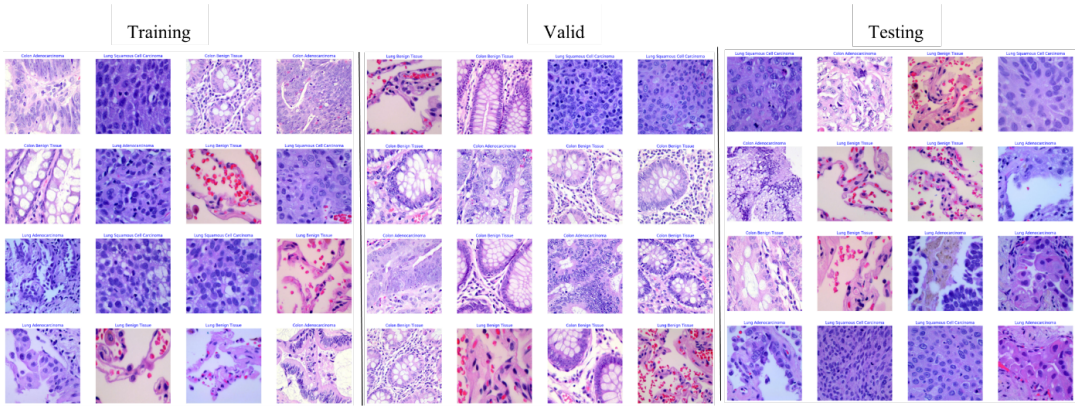


Figure 21. Example LC25000 images in the training, validation and testing splits. Source: SICET 2025 paper.

8.3 Benign and Malignant Tissue Classification

The task is a single five-class problem rather than separate benign-vs-malignant classifiers, so the model must also tell organs apart and separate the two lung carcinoma subtypes. Adenocarcinoma shows gland-forming structures; squamous cell carcinoma shows keratinisation and intercellular bridges; benign tissue shows regular architecture. The lung adenocarcinoma vs squamous cell boundary is the hardest pair, and it is where weaker models lose most accuracy.

8.4 Image Preprocessing and Data Augmentation

The preprocessing pipeline explored:

1. Resize to 768 × 768, grayscale copies and normalisation.
2. Contrast enhancement: CLAHE (contrast-limited adaptive histogram equalisation), histogram equalisation and contrast boosting.
3. Noise filtering: Gaussian, median and bilateral filters; Laplacian and unsharp masking for edges.
4. Segmentation: thresholding, edge detection, region growing and watershed.
5. Augmentation on training images: rotation, horizontal and vertical flips, zoom, brightness changes, cropping and colour mapping (virtual staining), via Keras ImageDataGenerator with batch size 64.
6. Model input: images resized to 224 × 224 × 3 and scaled for each backbone.

Not every transform suits histopathology: strong rotation, zoom or brightness changes can produce near-blank tiles, so augmentation strength must be checked visually before training.

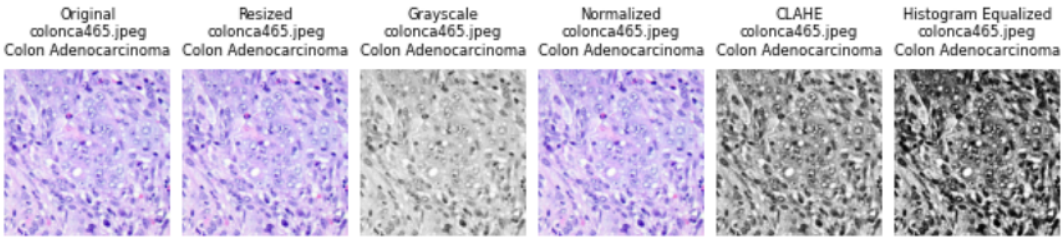


Figure 22. Preprocessing on a colon adenocarcinoma image: resizing, grayscale, normalisation, CLAHE and histogram equalisation. Source: SICET 2025 paper.

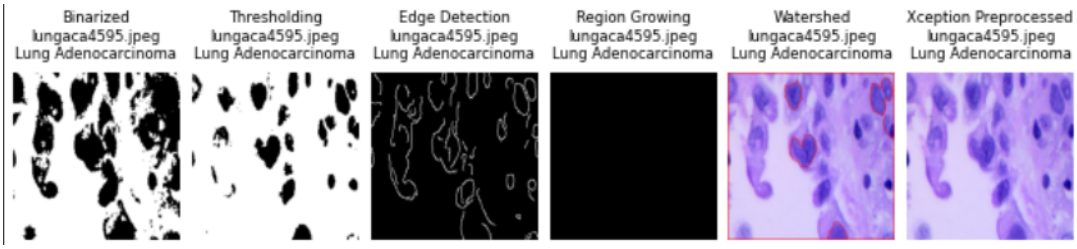


Figure 23. Segmentation outputs (binarisation, thresholding, edge detection, region growing, watershed) on lung adenocarcinoma. Source: SICET 2025 paper.

8.5 CNN Architectures for Histopathological Images

Table 24. CNN Architectures for Histopathological Images

Model	Backbone summary	Head	Dropout
Custom CNN	13 convolution layers (64→512 filters) in 5 blocks with max pooling	Dense 256 → Dense 64 → softmax 5	0.3
VGG16	13 × 3×3 convolutions in 5 blocks	GAP → Dense 256 → Dense 64 → softmax 5	0.5
ResNet50	7×7 stem + residual stages of 3, 4, 6, 3 bottleneck blocks	GAP → Dense 256 → Dense 64 → softmax 5	0.5
MobileNetV2	17 inverted-residual bottleneck blocks	GAP → Dense 128 → softmax 5	0.4
EfficientNetB0	MBConv stages, Swish activation, 1,280-channel final conv	GAP → Dense 256 → softmax 5	0.5
InceptionV3	Stem + Inception-A, B, C modules with grid reductions	GAP → Dense 256 → softmax 5	0.5
Xception	Entry, 8 middle-flow and exit blocks of depthwise separable convolutions	GAP → Dense 256 → softmax 5	None
Swin-Tiny	4×4 patch embedding; 4 stages (2, 2, 6, 2 blocks) of shifted-window attention	GAP → Dense 256 → softmax 5	0.5

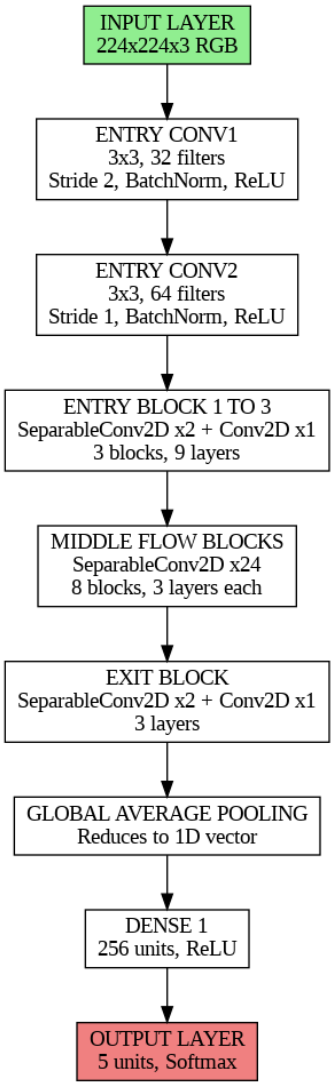


Figure 24. Layer structure of the Xception model with the classification head used in this study. Source: model architecture document.

8.6 Transfer Learning and Feature Extraction

All models except the custom CNN were initialised with ImageNet weights. Their pretrained base layers were frozen, so each backbone acted as a fixed feature extractor and only the new dense head was trained. This keeps training short and reduces overfitting, but it limits how far the features can adapt to stained tissue; partial fine-tuning of the top blocks is a natural next step.

8.7 Model Architecture and Training Procedure

Table 25. Model Architecture and Training Procedure

Setting	Value (all models)
Framework	Python, TensorFlow / Keras
Input size	224 × 224 × 3
Optimiser	Adamax, learning rate 0.001
Loss	Categorical cross-entropy
Batch size	64
Epochs	10, with early stopping on validation loss (patience 5)

Setting	Value (all models)
Pretrained weights	ImageNet (custom CNN from scratch)

The same hyperparameters were used for every model, which isolates the effect of architecture and makes the comparison fair. Model-specific tuning, for example by Bayesian optimisation, is left for future work.

8.8 Classification Performance and Evaluation Metrics

Table 26. Classification Performance and Evaluation Metrics

Model	Train acc.	Val. acc.	Test acc.	Sensitivity	Specificity	AUC	F1	Training time (s)
Xception	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1,200.23
ResNet50	0.9973	0.9927	0.9944	0.995	0.990	0.998	0.997	1,000.51
Custom CNN	0.9948	0.9819	0.9832	0.980	0.985	0.990	0.990	500.32
EfficientNetB0	0.9690	0.9715	0.9643	0.960	0.968	0.975	0.980	600.36
Swin-Tiny	0.9745	0.9602	0.9610	0.950	0.960	0.970	0.974	1,500.95
VGG16	0.9573	0.9527	0.9516	0.945	0.950	0.962	0.972	800.03
MobileNetV2	0.8697	0.8700	0.8644	0.850	0.860	0.880	0.915	300.22
InceptionV3	0.6629	0.6664	0.6535	0.640	0.670	0.700	0.725	700.41

Training and test accuracies are close for every model (largest gap 0.0135 for Swin-Tiny), so none shows classic overfitting. MobileNetV2 trained fastest (300 s) and Swin-Tiny slowest (1,501 s).

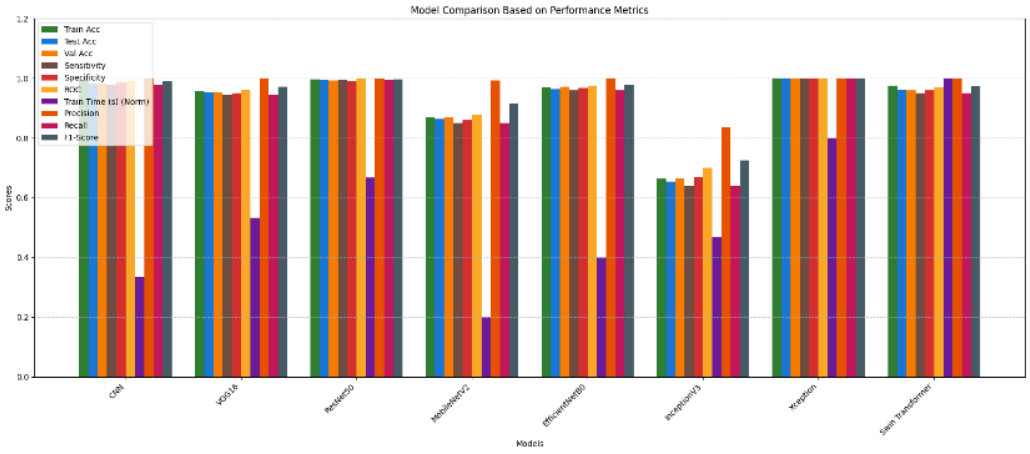


Figure 25. Comparison of the eight models on test performance metrics. Source: SICET 2025 paper.

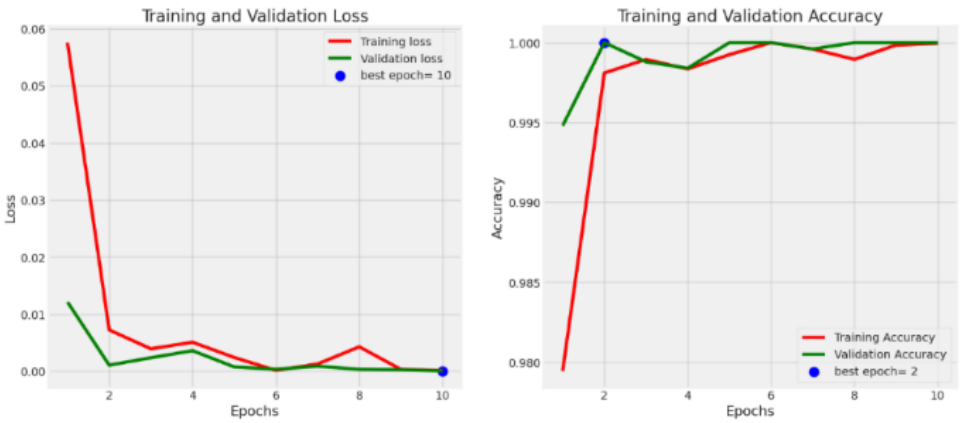


Figure 26. Training and validation loss (left) and accuracy (right) of the Xception model. Source: SICET 2025 paper.

8.9 Confusion Matrix, Precision, Recall and F1-Score

Table 27. Per-class precision / recall / F1 on the test set

Class	Xception	ResNet50	Custom CNN	EfficientNetB0	MobileNetV2	InceptionV3
Colon adenocarcinoma	1.00 / 1.00 / 1.00	0.99 / 1.00 / 1.00	0.99 / 1.00 / 0.99	0.96 / 1.00 / 0.98	0.87 / 0.78 / 0.82	0.68 / 0.49 / 0.57
Colon benign	1.00 / 1.00 / 1.00	1.00 / 0.99 / 1.00	1.00 / 0.99 / 0.99	1.00 / 0.97 / 0.99	0.85 / 0.88 / 0.87	0.62 / 0.73 / 0.67
Lung adenocarcinoma	1.00 / 1.00 / 1.00	0.98 / 1.00 / 0.99	0.95 / 0.98 / 0.97	0.97 / 0.88 / 0.92	0.82 / 0.82 / 0.82	0.64 / 0.60 / 0.62
Lung benign	1.00 / 1.00 / 1.00	1.00 / 1.00 / 1.00	1.00 / 0.99 / 1.00	0.98 / 1.00 / 0.99	0.97 / 0.96 / 0.97	0.64 / 0.66 / 0.65
Lung squamous cell carcinoma	1.00 / 1.00 / 1.00	1.00 / 0.98 / 0.99	0.98 / 0.96 / 0.97	0.91 / 0.97 / 0.94	0.82 / 0.88 / 0.85	0.76 / 0.85 / 0.80

Benign lung tissue is the easiest class for every model. Errors concentrate on the cancerous classes, especially lung adenocarcinoma, where EfficientNetB0's recall drops to 0.88 while its squamous-cell precision drops to 0.91 — the two subtypes are being confused.

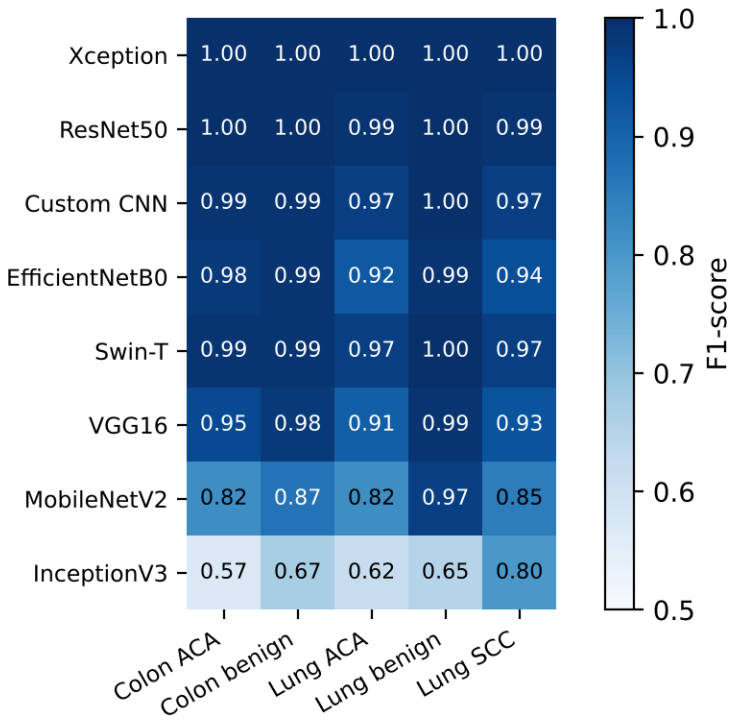


Figure 27. Per-class F1-score of each model on the LC25000 test set.

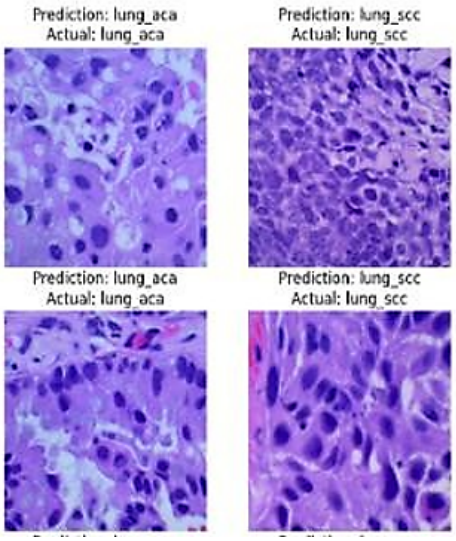


Figure 28. Example Xception predictions on lung adenocarcinoma and squamous cell carcinoma tiles. Source: SICET 2025 paper.

8.10 Comparison of Model Results

Table 28. Comparison of Model Results

Study	Method	Reported accuracy
Mangal et al. (2020)	Shallow CNN, RMSprop	97.9% lung, 96.6% colon (validation)
Hadiyoso et al. (2023)	VGG16 + CLAHE	98.96%
Al-Jabbar et al. (2023)	Modified Swin Transformer	99.80%
Tummala et al. (2023)	Hybrid fused features	99.64%
This study	Xception, frozen backbone + dense head	100%
This study	ResNet50, frozen backbone + dense head	99.44%

Xception's depthwise separable convolutions extract fine texture with few parameters, which suits nuclear and glandular patterns. InceptionV3's poor result (0.654) under the identical frozen-backbone protocol is unusual — published InceptionV3 results on LC25000 exceed 95% — and suggests a preprocessing or input-scaling mismatch for that model rather than a property of the architecture.

8.11 Overfitting, Data Leakage and Generalization

Near-perfect scores are common on LC25000, and the dataset's construction explains why. Its 25,000 images come from only 1,250 originals; rotated and flipped copies of the same tissue can fall into both the training and test sets under a random split. A model can then recognise a test image as a transformed copy of a training image instead of generalising.

To make the result robust:

- Group images by their source tile (for example with perceptual hashing) and split by group.
- Validate externally on an independent dataset such as the colon slides of NCT-CRC-HE-100K or lung cases from TCGA.
- Report confidence intervals with repeated runs or bootstrapping, and fix random seeds.
- Add Grad-CAM maps to confirm that predictions rely on tissue structure rather than staining artefacts.

8.12 Clinical Limitations and Future Research

- LC25000 tiles are small, pre-selected regions; real whole-slide images contain mixed tissue, artefacts and stain variation.
- Only five classes are covered; real practice includes many more subtypes and grades.
- Frozen ImageNet backbones were not fine-tuned; partial fine-tuning and stain normalisation may improve robustness on new data.
- Future work: grouped splits, external validation, whole-slide inference with patch aggregation, Grad-CAM explanations reviewed by a pathologist, and lightweight deployment of the best model.

Conclusion: under one fair protocol, Xception and ResNet50 were the strongest backbones for LC25000, while MobileNetV2 offered the best speed. Because of dataset-level near-duplicates, perfect test accuracy should be reported together with a leakage-aware split before it is used to claim clinical readiness.

09 CASE STUDY V
Machine Learning for Google Stock Analysis

KEY RESULT

Five hybrid forecasters, each blending a recurrent network (60%) with a tree or kernel model (40%), predicted GOOGL's daily close with test R^2 of about 0.98 but test MAPE of 38–162%. The gap between those two numbers is the lesson of this case study: high R^2 on raw price levels is easy, and the test period crosses the July 2022 20-for-1 stock split. Source: Google-stock-analysis, two Colab notebooks (baseline and hyperparameter-tuned).

9.1 Introduction to Financial Time-Series Analysis

A financial time series is a sequence of prices or returns ordered in time. Unlike medical images, it is non-stationary (its mean and variance change), noisy, and driven by information outside the data. The efficient-market hypothesis argues that past prices alone cannot reliably beat the market; ML studies therefore aim at short-horizon forecasting and at learning how models behave on sequential data.

- Problem: forecast the next trading day's closing price of Alphabet Class A shares (GOOGL).
- Objectives: explore price, volume and technical indicators; build hybrid deep-learning and machine-learning forecasters; measure the effect of hyperparameter tuning.

9.2 Google Stock Market Data Collection

Daily prices were loaded from a CSV file (googl_daily_prices.csv) with six fields: date, open, high, low, close and volume. The series runs from about the 2004 IPO to 27 May 2025 — roughly 5,200 trading days; after a 200-day indicator warm-up and 30-day windowing, about 5,000 sequences remain. Rows with missing values or non-positive prices were removed and the data sorted by date.

The file holds unadjusted prices. The close drops from about $1,200 to $560 in April 2014 (share-class split) and from about $2,250 to $112 in July 2022 (20:1 split). Using split-adjusted prices is essential for modelling and is the first correction recommended in Section 9.12.

9.3 Historical Price and Trading-Volume Analysis

Close prices were plotted with 50-day and 200-day moving averages to show long-term trend, a candlestick chart displayed the last 100 trading days, and volume was plotted as bars. Over those last 100 days the close ranged from $144.70 to $206.38, opening the window at $189.30 and closing it at $172.90. Daily high–low range was plotted as a simple volatility proxy; it peaked near $185 in late 2021 before the split.

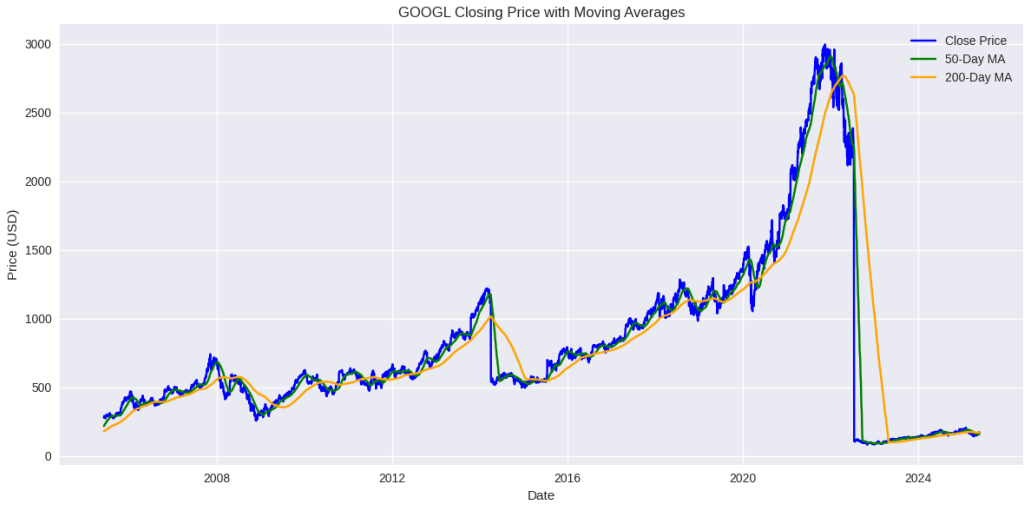


Figure 29. GOOGL closing price with 50-day and 200-day moving averages (unadjusted prices; note the 2014 and 2022 splits). Source: Google-stock-analysis repository.

9.4 Exploratory Data Analysis and Visualization

Table 29. Exploratory Data Analysis and Visualization

Pair	Pearson correlation
Open, high, low, close (with each other)	1.00
Volume vs prices	−0.52
RSI vs prices	0.07
Volume vs RSI	−0.07

The four price fields are perfectly correlated, so using them together adds no information. The negative volume–price correlation reflects the unadjusted data: share counts rose after each split while nominal prices fell. Bollinger Band and RSI plots with 70/30 overbought–oversold lines completed the EDA. A fuller analysis would add daily returns, rolling standard deviation of returns, and stationarity tests (augmented Dickey–Fuller).

9.5 Feature Engineering for Stock Data

Table 30. Feature Engineering for Stock Data

Feature	Definition
MA50, MA200	50- and 200-day simple moving averages of close
RSI(14)	Relative strength index from 14-day average gains and losses
Bollinger upper / lower	20-day mean ± 2 standard deviations
prev_close	Previous day's close
Volume	Daily traded shares

Two input designs were used. The deep-learning branch takes the last 30 min–max-scaled closes as a sequence. The machine-learning branch takes the seven indicator features above for the same day.

9.6 Regression-Based Stock Prediction

Each of the five models is a fixed-weight hybrid: prediction = 0.6 × deep model + 0.4 × machine-learning model.

Table 31. Regression-Based Stock Prediction

Hybrid	Deep component	ML component
H1	LSTM	Random Forest
H2	GRU	XGBoost
H3	CNN-LSTM	Support vector regression (RBF)
H4	LSTM	Gradient Boosting
H5	GRU	LightGBM

Baseline deep models stack two recurrent layers of 50 units with dropout 0.2 and a dense output; CNN-LSTM adds a Conv1D layer (64 filters, kernel 3) in front. All use Adam (learning rate 0.001), MSE loss, 20 epochs and batch size 32. Tree models use 100 estimators.

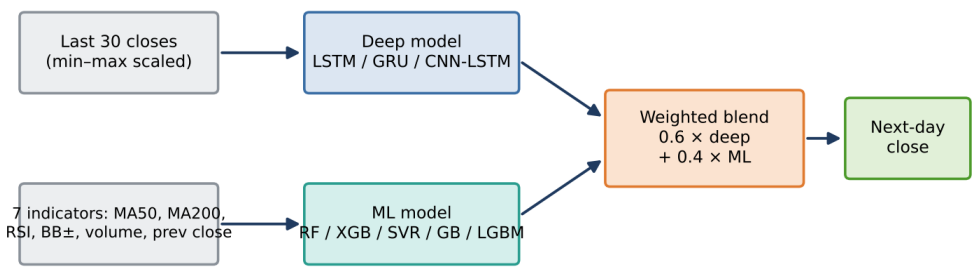


Figure 30. Design of the hybrid forecasters: a deep sequence model and an indicator-based ML model blended 60:40.

9.7 Time-Series Forecasting Concepts

Key concepts for this task:

- Stationarity: prices trend; returns or log-returns are closer to stationary and are the usual modelling target.
- Sliding windows: the last n observations form one input sample (n = 30 here).
- Forecast horizon: one step ahead here; multi-step forecasts accumulate error.
- Naive baseline: predicting that tomorrow's close equals today's is surprisingly hard to beat and must always be reported.
- Classical models: ARIMA and exponential smoothing provide statistical baselines; LSTM and GRU learn non-linear temporal patterns.

9.8 Chronological Training and Testing

The final evaluation used a chronological 80/20 split: the earliest 80% of sequences for training and the latest 20% (about mid-2021 to May 2025) for testing. This correctly prevents the model from training on the future.

The tuning stage did not keep that discipline. Deep-model tuning used shuffled 3-fold KFold, and the tree models used GridSearchCV's default folds, so some validation folds came from before the training data. TimeSeriesSplit, which always validates on a later block, is the correct choice.

9.9 Hyperparameter Tuning and Model Optimization

Table 32. A full grid search was run for every component

Component	Search space	Best values
LSTM	units {50, 100}, dropout {0.2, 0.3}, lr {1e-3, 1e-4}	50 units, dropout 0.3, lr 1e-3

Component	Search space	Best values
GRU	same	100 units, dropout 0.3, lr 1e-4
CNN-LSTM	same + filters {32, 64}	100 units, dropout 0.3, lr 1e-3, 32 filters
Random Forest	n_estimators {100, 200}, max_depth {10, 20, None}	100 trees, no depth limit
XGBoost	n_estimators {100, 200}, max_depth {3, 6}, lr {0.01, 0.1}	200 trees, depth 3, lr 0.1
SVR	C {0.1, 1, 10}, epsilon {0.01, 0.1}	C 10, epsilon 0.01
Gradient Boosting	n_estimators {100, 200}, max_depth {3, 5}, lr {0.01, 0.1}	200 trees, depth 5, lr 0.1
LightGBM	n_estimators {100, 200}, max_depth {10, 20}, lr {0.01, 0.1}	100 trees, depth 20, lr 0.1

9.10 Evaluation Using MAE, MSE, RMSE and Other Relevant Metrics

Table 33. Test-set results in US dollars (baseline run vs tuned run)

Hybrid	Baseline RMSE	Tuned RMSE	Baseline MAE	Tuned MAE	Tuned R^2	Tuned MAPE
GRU + XGBoost	144.10	128.88	113.74	88.40	0.987	43.8%
GRU + LightGBM	143.81	132.84	107.39	89.71	0.986	43.2%
LSTM + Random Forest	147.66	149.39	103.51	106.30	0.983	50.1%
LSTM + Gradient Boosting	143.72	152.63	93.32	110.08	0.982	53.0%
CNN-LSTM + SVR	476.90	414.35	412.64	351.77	0.866	134.3%

Training RMSE was about $18–26 for the four strong hybrids, against $129–153 on test: the models fit the pre-2021 regime closely but transfer poorly to a period that includes the 2022 split. The CNN-LSTM + SVR hybrid was worst because the SVR received unscaled features, so trading volume (in millions) dominated its kernel.

R^2 and MAPE tell opposite stories. R^2 stays near 0.98 because the test period spans a jump from about $2,500 to about $150, so almost any model that follows the level explains most of the variance. MAPE of 40–50% shows that after the split the hybrids predicted about $200–250 against true prices of $100–200 — tree models cannot predict outside the range of targets they saw in training.

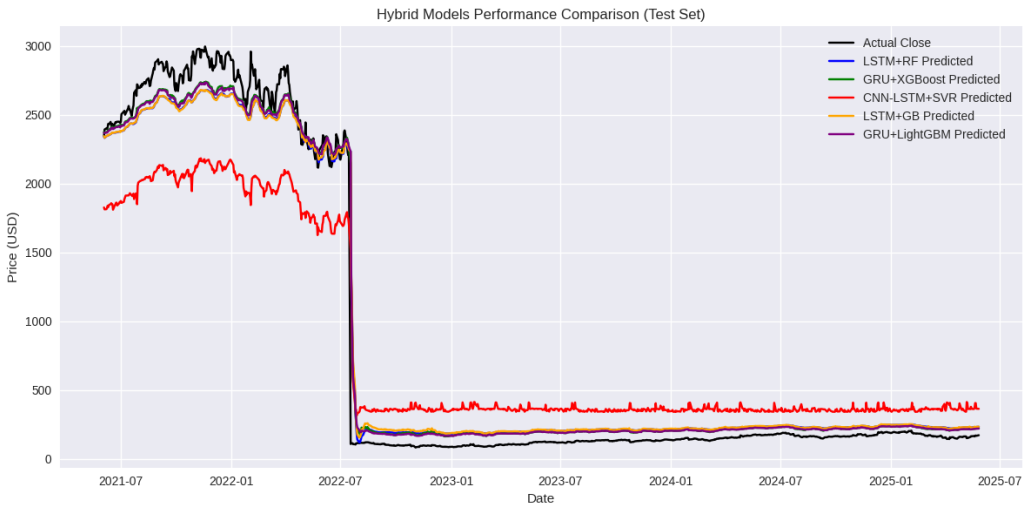


Figure 31. Actual versus predicted close for the five hybrids on the test period; predictions stay above $200 after the 2022 split. Source: Google-stock-analysis repository.

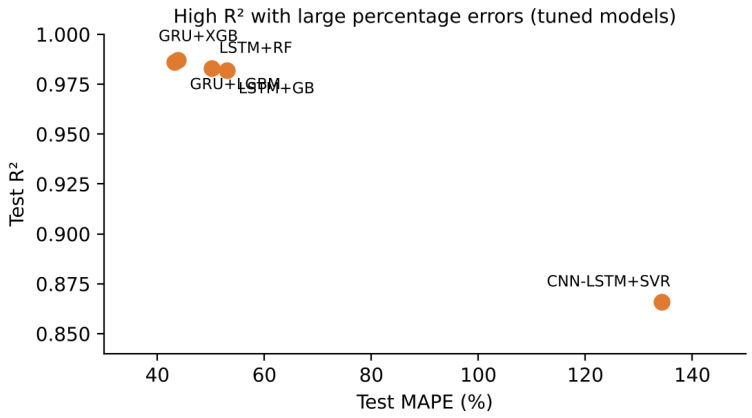


Figure 32. Test R^2 versus test MAPE for the tuned hybrids: high R^2 coexists with large percentage errors.

9.11 Comparison of Baseline and Optimized Models

Tuning lowered test RMSE for three hybrids (GRU + XGBoost by \$15, GRU + LightGBM by \$11, CNN-LSTM + SVR by \$63) and raised it for two LSTM hybrids. The improvement is within run-to-run noise: across three untuned runs, the GRU + LightGBM test RMSE alone varied from \$108.83 to \$143.81, and no TensorFlow seed was fixed. The fair conclusion is that tuning did not produce a measurable gain under this setup.

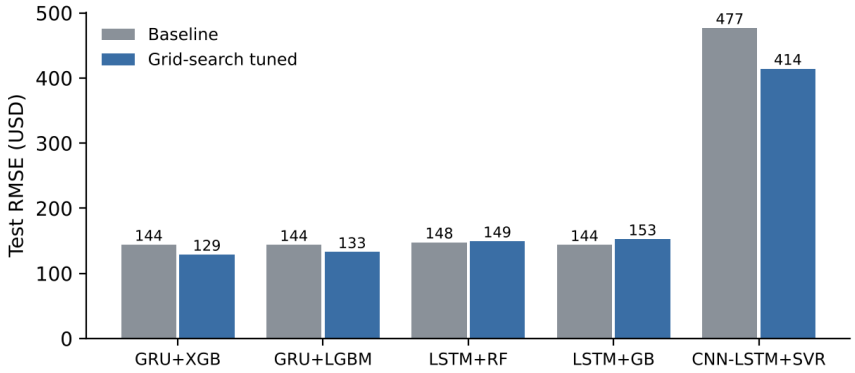


Figure 33. Test RMSE of the five hybrids before and after grid-search tuning.

9.12 Limitations of Stock Price Prediction

Table 34. Limitations of Stock Price Prediction

Issue	Effect	Fix
Unadjusted prices across the 2014 and 2022 splits	Inflates R^2, breaks tree models after 2022	Use split-adjusted close or predict returns
Indicators (MA, RSI, Bollinger) include day t's close	Target leakage in the ML branch	Shift all indicators by one day
Min–max scaler fitted on the full series	Test information leaks into training	Fit scaler on training data only
Shuffled folds in tuning	Validation sees the future	Use TimeSeriesSplit
Unscaled SVR inputs	Volume dominates the kernel	Standardise features
Fixed 0.6 / 0.4 blend	Weights never validated	Learn weights by stacking on a validation block
No seeds, no naive baseline	Results not reproducible or benchmarked	Fix seeds; report persistence and ARIMA baselines

Beyond these fixes, daily prices respond to earnings, regulation and macro news that no price-only model can see. Even a correct model should be judged on returns, directional accuracy and a trading backtest with costs, not on price-level R^2.

Conclusion: the project demonstrates a complete forecasting workflow — indicators, recurrent networks, tree ensembles, hybrids and grid search — and, more valuably, shows how data errors (unadjusted splits) and subtle leakage can make metrics look excellent while forecasts are poor.

10 Comparative Analysis Across the Five Projects

Across the five projects, the factor that most changed the reported result was not the model but the evaluation protocol: leakage-aware splits cut a fungi probe's accuracy by 12 points, unadjusted stock splits made R^2 look excellent while MAPE exceeded 40%, and repeated splits showed PCOS synthetic-data gains to be within noise.

10.1 Comparison of Healthcare and Financial Applications

Table 35. Comparison of Healthcare and Financial Applications

Aspect	Healthcare projects (I–IV)	Financial project (V)
Task	Classification	Regression / forecasting
Label stability	Stationary: a tissue pattern means the same thing next year	Non-stationary: market regimes shift
Cost of error	Missed disease (false negative)	Financial loss; overconfidence
Main metric	Macro-F1, recall, ROC-AUC	RMSE, MAE, MAPE, directional accuracy
Main leakage risk	Same patient or slide in train and test	Future data used to predict the past
Ground truth	Expert diagnosis	Observed price

10.2 Structured Data Versus Image Data Versus Time-Series Data

Table 36. Structured Data Versus Image Data Versus Time-Series Data

Project	Data type	Samples	Features / input	Key preprocessing
I. PCOS	Tabular	541 + 172	41 clinical / 34–39 survey features	Type fixing, imputation, encoding, scaling
II. Fungi	Microscopy images	6,802	224 × 224 RGB + 24 morphometrics	Crops, eight-way rotation, segmentation
III. Skin	Clinical photographs	Public datasets (e.g. HAM10000, DermNet)	224 × 224 RGB	Resize, colour and brightness augmentation
IV. Colon–lung	Histopathology images	25,000	224 × 224 RGB	CLAHE, filtering, augmentation
V. Stock	Daily time series	\~5,200 days	30-day windows + 7 indicators	Indicators, windowing, scaling

Tabular data rewards careful cleaning and feature understanding; image data rewards augmentation and pretrained representations; time-series data rewards strict respect for time order.

10.3 Comparison of Machine Learning and Deep Learning Approaches

Classical ML matched or beat deep learning when data were small and tabular: on the PCOS clinical cohort, random forest, logistic regression and XGBoost reached ROC-AUC 0.946 without any synthetic augmentation, and no generator improved on them. Deep learning was essential for images, where pretrained CNNs reached 83–100% accuracy that handcrafted features (69.7% for fungal morphometrics alone) could not approach.

Hybrids did not automatically help. Fusing morphometrics with DINOv2 lowered fungi accuracy by 3.2 points, and blending recurrent and tree models for stock prices inherited the weaknesses of both. A hybrid needs evidence, from an ablation, that each part contributes.

10.4 Model Selection and Computational Requirements

Table 37. Model Selection and Computational Requirements

Project	Best model	Compute	Training time
I. PCOS	Baseline classifiers (clinical); TabDDPM-lite augmentation (survey)	CPU	Seconds to \~1 minute per generator
II. Fungi	ConvNeXtV2-Tiny (27.9 M parameters)	T4 GPU	12.4 minutes
III. Skin	Lightweight CNN (MobileNetV2 / EfficientNet-B0 class)	GPU to train; phone to run	—
IV. Colon–lung	Xception (22.9 M)	GPU	1,200 s; MobileNetV2 fastest at 300 s
V. Stock	GRU + XGBoost hybrid	CPU / GPU	Minutes

The best model is the one that fits the deployment constraint. On a phone, MobileNetV2 (3.5 M parameters) may be the right choice even though it trailed Xception by 13.6 points on LC25000.

10.5 Comparison of Evaluation Metrics

Table 38. Comparison of Evaluation Metrics

Project	Headline result	What the headline hides
I. PCOS	Clinical ROC-AUC 0.946	Survey F1 standard deviation up to ±0.17 on 9 test positives
II. Fungi	Accuracy 85.1%	1-NN probe loses 12 points under group-aware CV
IV. Colon–lung	Accuracy 100% (Xception)	LC25000 near-duplicates from 1,250 originals
V. Stock	Test R^2 0.98	Test MAPE 38–162% across the 2022 split

Every project shows the same pattern: a single summary number looked better than the full picture. Reporting variance, per-class metrics, calibration and a naive baseline closes that gap.

10.6 Interpretability and Explainability

Table 39. Interpretability and Explainability

Project	Method	What it revealed
I. PCOS	SHAP, LIME	Rotterdam-criteria features recovered without labels; moderate SHAP–LIME agreement (ρ 0.40–0.59)
II. Fungi	Grad-CAM, attention maps, gate values, volcano plots	ConvNeXtV2 attends to hyphae; MorphoFuse attention diffuse; BASH has almost no morphometric signature
III. Skin	Confidence display; planned Grad-CAM overlay	Supports referral decisions in the app
IV. Colon–lung	Recommended Grad-CAM	Not yet produced
V. Stock	Feature correlation	Price fields redundant; RSI nearly uncorrelated with price

10.7 Data Quality, Bias and Generalization

Every dataset came from a narrow source: one hospital region (PCOS), one laboratory (DeFungi), a few original slides (LC25000), one stock with unadjusted corporate actions. Data defects — non-numeric hormone values, a mislabelled height column, a leaking medication field, unadjusted splits — were found only by inspecting the raw files. None of the five projects has yet been validated on an independent external dataset, which is the most important step for generalisation.

10.8 Common Implementation Challenges

1. Dirty raw data that silently changes column types.
2. Leakage through preprocessing fitted before the split, through grouped samples, or through time.
3. Small or imbalanced classes that make single-split metrics unstable.
4. Compute limits: CPU-only Colab sessions made fine-tuning impractical and forced frozen-feature designs.
5. Notebook state: a cell that failed to load data still ran on stale variables in the stock project.
6. Reproducibility: missing seeds and unrecorded versions in some notebooks.

10.9 Lessons Learned Across the Projects

- Start with a strong simple baseline, and report it.
- Audit the data before modelling; most surprises are data problems.
- Choose the split to match how the model will be used: by patient, by slide, or forward in time.
- Report variance and confidence intervals, not single numbers.
- Prefer macro-F1, recall and calibration over accuracy in medicine; prefer returns and naive baselines over R^2 in finance.
- Publish negative results; they save other researchers time.
- Keep code, splits and results in a repository so others can reproduce and extend the work.

11 Ethical, Privacy and Responsible AI Considerations

A model that scores well in a notebook can still cause harm in use. Four of the five projects touch patient data and one touches investment decisions, so each carries responsibilities that go beyond accuracy.

11.1 Patient Data Privacy and Confidentiality

Health records and medical images are special-category personal data under laws such as the EU GDPR, the US HIPAA and Sri Lanka's Personal Data Protection Act No. 9 of 2022. Researchers should use de-identified data, remove identifiers such as patient file numbers, restrict access to raw files, and obtain ethics approval when data are collected rather than downloaded from a public source.

Synthetic data, as explored in the PCOS case study, can reduce disclosure risk, but it is not automatically private. Distance-to-closest-record checks must confirm that generated rows do not copy real patients, and formal guarantees require methods such as differential privacy. On-device inference, as in the skin disease app, keeps photographs on the user's phone.

11.2 Bias and Fairness in Healthcare Models

A model inherits the biases of its training data. The PCOS clinical cohort comes from hospitals in Kerala, India; skin datasets are often dominated by lighter skin tones; LC25000 was derived from a small number of source slides; DeFungi comes from a single laboratory. Performance on other populations, skin types, scanners or stains cannot be assumed.

Mitigations include reporting results per subgroup, collecting more diverse data, external validation on a second site, and documenting known gaps in a model card.

11.3 Reliability of Automated Medical Predictions

A classifier should support, not replace, a clinician. Reliable systems are calibrated (their confidence matches their accuracy), report uncertainty, and abstain or refer when confidence is low. Conformal prediction, used in the fungi study, offers prediction sets with a guaranteed coverage rate.

A high test accuracy on a curated benchmark is weak evidence of clinical value. Prospective evaluation on consecutive real cases, and regulatory review where required, come before any diagnostic use.

11.4 Financial Prediction Risks and Uncertainty

Stock prices react to news, policy and market sentiment that historical prices do not contain. A low error on next-day closing price often reflects the fact that tomorrow's price is close to today's, not genuine predictive skill. Forecasts should be compared with a naive "tomorrow equals today" baseline and presented with prediction intervals.

Models in this chapter are educational. They are not investment advice, and they ignore transaction costs, slippage and risk management.

11.5 Explainable and Trustworthy AI

Clinicians and regulators need to know why a model made a decision. SHAP and LIME explain tabular predictions (for PCOS they recover Rotterdam-criteria features such as follicle count, cycle irregularity and hair growth). Grad-CAM shows which image regions drove a CNN's output. An explanation that highlights background or slide artefacts is a warning sign of shortcut learning.

11.6 Reproducibility and Research Transparency

Transparent research publishes code, data splits, random seeds, hyperparameters and full results, including negative findings. The fungi study is a good example: it reports that morphology fusion did not beat a strong visual backbone, instead of hiding the result. Inflated results from leaky splits damage trust in the whole field.

11.7 Responsible Deployment and Human Oversight

Responsible deployment keeps a human in the loop for every consequential decision, logs predictions for audit, monitors performance drift after release, and gives users a clear way to report errors. Mobile health apps should state plainly that they are screening aids, show confidence levels, and direct users to a qualified professional.

12 Conclusion and Future Directions

The five implementations show that ML and DL can deliver useful screening and forecasting tools, but only when the evaluation is as carefully designed as the model.

12.1 Summary of the Five Implementations

Table 40. Summary of the Five Implementations

Project	Approach	Key result
I. PCOS	Eight synthetic-data generators; clustering, embeddings and anomaly detection; SHAP and LIME	No significant oversampling gain; clinical PCOS recoverable without labels (K-Means 0.810 balanced acc.; Isolation Forest AUC 0.890)
II. Fungi	ResNet50, ConvNeXtV2, DINOv2 and morphology-guided fusion, with leakage audit and calibration	ConvNeXtV2 85.1% accuracy, macro-F1 0.883; fusion did not help; 12-point optimism from slide-level leakage

Project	Approach	Key result
III. Skin	Transfer-learned CNN deployed on a phone via TensorFlow Lite	End-to-end offline screening workflow on a phone
IV. Colon–lung	Eight CNN and transformer backbones on LC25000	Xception 100%, ResNet50 99.4%; leakage-aware validation still needed
V. Stock	LSTM, GRU and CNN-LSTM blended with tree models; grid search	Test R^2 \~0.98 but MAPE 38–162%; unadjusted splits and indicator leakage identified

12.2 Key Findings and Contributions

1. Strong pretrained backbones dominate image tasks; handcrafted features and simple fusion added little.
2. On small tabular data, simple classical models are hard to beat, and synthetic data must be tested against an unaugmented baseline.
3. Leakage, in its group, temporal, preprocessing and target forms, was the most common threat to valid results across all five projects.
4. Unsupervised analysis can show whether a disease forms a distinct group (clinical PCOS) or a departure from normal (survey PCOS).
5. Calibration, conformal prediction and clinically grouped error analysis turn a classifier into a decision-support tool.

12.3 Practical Benefits and Limitations

Benefits: low-cost PCOS screening from routine clinical and questionnaire data; faster triage of fungal microscopy and histopathology slides; offline skin screening on a phone; a reusable forecasting workflow for financial data.

Limitations: every project relies on a single-source dataset; none has external or prospective validation; and some results come from single seeds.

12.4 Future Research Opportunities

- External validation: DIFaS and SFMID for fungi; NCT-CRC-HE-100K or TCGA slides for colon and lung; a second hospital cohort for PCOS.
- Leakage-aware protocols as standard: grouped splits for images, TimeSeriesSplit for prices.
- Split-adjusted prices and return-based targets with a naive persistence baseline for the stock model.
- Partial fine-tuning, stain normalisation and longer training for the image models.
- Clinical studies with specialists to measure real diagnostic benefit.
- Privacy-preserving learning: differentially private synthetic data and federated training across hospitals.

12.5 Emerging Trends in Machine Learning and Deep Learning

- Foundation models for pathology and microscopy (UNI, CONCH, Virchow) pretrained on millions of slides, replacing ImageNet backbones.
- Diffusion models for tabular and image synthesis, as explored with TabDDPM and TabSyn in the PCOS study.
- Multimodal models combining images, clinical records and text reports.
- Agentic AI pipelines that automate data checks, experiment runs and reporting, with human oversight.
- Efficient on-device AI through quantisation, pruning and distillation, as in the skin app.
- Trustworthy AI regulation, such as the EU AI Act, which classifies medical AI as high-risk and requires documentation, transparency and human oversight.

13 References and Supplementary Materials

References are grouped by purpose.

13.1 Academic References and Related Research

FOUNDATIONS

1. Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill.
2. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521, 436–444.
3. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
4. Breiman, L. (2001). Random forests. *Machine Learning*, 45, 5–32.
5. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proc. KDD*.
6. Ke, G., et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. *NeurIPS*.
7. Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM TKDD*, 6(4).

ARCHITECTURES

1. Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. *ICLR*.
2. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *CVPR*.
3. Szegedy, C., et al. (2016). Rethinking the Inception architecture for computer vision. *CVPR*.
4. Chollet, F. (2017). Xception: Deep learning with depthwise separable convolutions. *CVPR*.
5. Huang, G., Liu, Z., van der Maaten, L., & Weinberger, K. Q. (2017). Densely connected convolutional networks. *CVPR*.
6. Sandler, M., et al. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. *CVPR*.
7. Tan, M., & Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. *ICML*.
8. Vaswani, A., et al. (2017). Attention is all you need. *NeurIPS*.
9. Dosovitskiy, A., et al. (2021). An image is worth 16×16 words: Transformers for image recognition at scale. *ICLR*.
10. Liu, Z., et al. (2021). Swin Transformer: Hierarchical vision transformer using shifted windows. *ICCV*.
11. Woo, S., et al. (2023). ConvNeXt V2: Co-designing and scaling ConvNets with masked autoencoders. *CVPR*.
12. Oquab, M., et al. (2024). DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*.
13. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
14. Cho, K., et al. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. *EMNLP*.

SYNTHETIC DATA, IMBALANCE AND UNSUPERVISED LEARNING

1. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *JAIR*, 16, 321–357.
2. He, H., Bai, Y., Garcia, E. A., & Li, S. (2008). ADASYN: Adaptive synthetic sampling approach for imbalanced learning. *IJCNN*.
3. Xu, L., Skoularidou, M., Cuesta-Infante, A., & Veeramachaneni, K. (2019). Modeling tabular data using conditional GAN. *NeurIPS*.
4. Kotelnikov, A., Baranchuk, D., Rubachev, I., & Babenko, A. (2023). TabDDPM: Modelling tabular data with diffusion models. *ICML*.
5. Zhang, H., et al. (2024). Mixed-type tabular data synthesis with score-based diffusion in latent space (TabSyn). *ICLR*.
6. Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. *IEEE ICDM*.
7. van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *JMLR*, 9, 2579–2605.
8. McInnes, L., Healy, J., & Melville, J. (2018). UMAP: Uniform manifold approximation and projection. arXiv:1802.03426.

EXPLAINABILITY AND RELIABILITY

1. Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *NeurIPS*.
2. Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?" Explaining the predictions of any classifier. *KDD*.
3. Selvaraju, R. R., et al. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. *ICCV*.
4. Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. *ICML*.
5. Angelopoulos, A. N., & Bates, S. (2023). Conformal prediction: A gentle introduction. *Foundations and Trends in Machine Learning*, 16(4).

APPLICATION LITERATURE CITED IN THE CASE STUDIES

1. Rotterdam ESHRE/ASRM-Sponsored PCOS Consensus Workshop Group. (2004). Revised 2003 consensus on diagnostic criteria and long-term health risks related to PCOS. *Human Reproduction*, 19(1), 41–47.
2. Pineda Sopo, C., Hajati, F., & Gheisari, S. (2021). DeFungi: Direct mycological examination of microscopic fungi images. arXiv: 2109.07322.
3. Sajjadi et al. (2025). Few-shot graph neural network classification of DeFungi images. *Biomedical Signal Processing and Control*.
4. Esteva, A., et al. (2017). Dermatologist-level classification of skin cancer with deep neural networks. *Nature*, 542, 115–118.
5. Tschandl, P., Rosendahl, C., & Kittler, H. (2018). The HAM10000 dataset. *Scientific Data*, 5, 180161.
6. Borkowski, A. A., et al. (2019). Lung and colon cancer histopathological image dataset (LC25000). arXiv:1912.12142.
7. Mangal, S., Chaurasia, A., & Khajanchi, A. (2020). Convolution neural networks for diagnosing colon and lung cancer histopathological images. arXiv:2009.03878.
8. Hadiyoso, S., Aulia, S., & Irawati, I. D. (2023). Diagnosis of lung and colon cancer based on clinical pathology images using CNN and CLAHE. *IJASE*, 20(1).
9. Al-Jabbar, M., Alshahrani, M., Senan, E. M., & Ahmed, I. A. (2023). Histopathological analysis for detecting lung and colon cancer malignancies. *Bioengineering*, 10(3), 383.
10. Tummala, S., Kadry, S., Nadeem, A., Rauf, H. T., & Gul, N. (2023). An explainable classification method based on complex scaling in histopathology images for lung and colon cancer. *Diagnostics*, 13(9), 1594.

AUTHOR'S OWN WORK

1. Karunakaran, L. (2025). Deep learning based multi-class classification model for lung and colon cancer detection. *SLIIT International Conference on Engineering and Technology (SICET 2025)*, Malabe, Sri Lanka.

13.2 Dataset Citations and Usage Licenses

Table 41. Dataset Citations and Usage Licenses

Dataset	Citation	Access	Licence note
PCOS clinical (541 patients)	Kottarathil, P. (2020). Polycystic ovary syndrome (PCOS). Kaggle	Kaggle	Check Kaggle page licence
PCOS lifestyle survey (173 respondents)	De-identified questionnaire data	Not public	Used with permission
DeFungi	Pineda Sopo et al. (2021); Kaggle repack "Microscopic Fungi Images"	Kaggle / UCI	Check repack licence
LC25000	Borkowski et al. (2019)	Kaggle, GitHub	Released for research use
Skin images	Tschandl et al. (2018), HAM10000 / ISIC	ISIC archive	CC BY-NC 4.0 (non-commercial)
GOOGL daily prices	Yahoo Finance or equivalent provider	CSV	Provider terms of use

13.3 GitHub Repository References

Table 42. GitHub Repository References

Project	Repository	Contents	Access
I. PCOS	Implementation-in-PCOS	PCOD.ipynb, PCOS_Unsupervised.ipynb, result CSVs, 20 figures	Public
II. Fungi	Fung-implementation	MorphoFuse notebooks and results	On request
III. Skin	skin-disease-detection-mobile-app	Model training and mobile app	On request
IV. Colon–lung	Colon-lung-cancer-image-data	SICET 2025 paper, architecture and result documents	Public
V. Stock	Google-stock-analysis	Baseline and hyperparameter-tuning notebooks	Public

13.4 Software Libraries and Frameworks

Table 43. Software Libraries and Frameworks

Library	Version recorded	Used in
Python	3.12 (Colab)	All
pandas / NumPy	2.2.2 / 2.0.2	All
scikit-learn	1.6.1	I, V
imbalanced-learn	0.14.2	I
ctgan	0.12.1	I
XGBoost	3.3.0 (I); 2.1.4 (V)	I, V
LightGBM	4.5.0	V
PyTorch	2.11	I, II
timm	1.0.29	II
TensorFlow / Keras	2.18.0 / 3.8.0 (V); version not recorded for IV	IV, V
SHAP, LIME, umap-learn	Not recorded	I
Matplotlib / Seaborn / Plotly	3.10.0 / 0.13.2 / 5.24.1	All

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *JMLR*, 12, 2825–2830. Paszke, A., et al. (2019). PyTorch. *NeurIPS*. Abadi, M., et al. (2016). TensorFlow. *OSDI*.

13.5 Supplementary Tables, Figures and Code

Notebooks, result tables (CSV) and full-resolution figures for every case study are available in the repositories listed in Section 13.3. Figures reproduced from the repositories and from the SICET 2025 paper are credited in their captions; all other diagrams and charts were drawn for this chapter from the reported results.